

AWS production troubleshooting

06/06/2026 17:36:05

<https://claude.ai/chat/4bff8ac2-7c0e-4d89-a605-5293a6611a0d>

AWS all interview Questions and answers after adding above those add thes...

Now I'll write the comprehensive AWS Q&A document. This is a large...

dont create document write everything here

Got it! Here's everything inline. --- # AWS Interview Questions & Answers --- #...

give at least 10 each sections production issues on EC2, s3, RDS, Lamda, ASG...

AWS Production Issues — Deep Dive --- # EC2 Production Issues --- **Issue...



AWS all interview Questions and answers
after adding above those
add these then
aws productions issues Q&A (with how to fix + prevention)

6/5/2026, 10:24:49 AM



Viewed file ▾

📄 Check skill for docx creation

👍 Done

Now I'll write the comprehensive AWS Q&A document. This is a large document —

6/5/2026, 10:24:57 AM



dont create document write everything here

6/5/2026, 10:25:13 AM



Got it! Here's everything inline.

AWS Interview Questions & Answers

1. Core Fundamentals

Q: What is AWS and what are its core benefits?

AWS (Amazon Web Services) is a cloud platform offering 200+ services across compute, storage, networking, databases, ML, and more. Core benefits: pay-as-you-go pricing, global infrastructure, elasticity, high availability, managed services, and security compliance.

Q: What is the AWS Global Infrastructure?

It consists of Regions (geographic areas, e.g. ap-south-1), Availability Zones (isolated data centers within a region, minimum 2 per region), Edge Locations (CDN endpoints for CloudFront), and Local Zones (extensions of regions for ultra-low latency).

Q: What is the Shared Responsibility Model?

AWS is responsible for security **of** the cloud (hardware, facilities, hypervisor, managed service infrastructure). You are responsible for security **in** the cloud (OS patching, IAM, data encryption, network config, application security).

Q: What is an AMI?

An Amazon Machine Image is a template for launching EC2 instances. It includes the OS, application server, and applications. AMIs can be AWS-provided, marketplace, or custom-built and are region-specific (but can be copied).

2. EC2 (Elastic Compute Cloud)

Q: What are EC2 instance types?

- **General Purpose** (t3, m6i) — balanced CPU/memory
 - **Compute Optimized** (c6i) — high CPU, batch processing
 - **Memory Optimized** (r6i, x2i) — in-memory DBs, big data
 - **Storage Optimized** (i3, d3) — high IOPS, data warehousing
 - **Accelerated Computing** (p4, g5) — GPU, ML workloads
-

Q: What are EC2 purchasing options?

- **On-Demand** — pay per second, no commitment
 - **Reserved Instances** — 1 or 3 year commitment, up to 72% savings
 - **Spot Instances** — bid on unused capacity, up to 90% savings, can be interrupted
 - **Savings Plans** — flexible commitment to usage amount (\$/hr)
 - **Dedicated Hosts** — physical server dedicated to you, for compliance/licensing
-

Q: What is the difference between Stop, Terminate, and Hibernate?

Stop: instance shuts down, EBS root volume persists, no charge for compute. Terminate: instance and root EBS volume deleted (unless DeleteOnTermination=false). Hibernate: RAM state saved to EBS, instance resumes faster, useful for long-running processes.

Q: What are Security Groups vs NACLs?

Security Groups are stateful (return traffic automatically allowed), operate at instance level, only allow rules. NACLs (Network ACLs) are stateless (must explicitly allow inbound AND outbound), operate at subnet level, support both allow and deny rules. NACLs are evaluated in rule number order; Security Groups evaluate all rules together.

Q: What is EC2 Auto Scaling?

It automatically adjusts EC2 capacity based on demand. Components: Launch Template (what to launch), Auto Scaling Group (min/max/desired capacity), Scaling Policies (when to

scale). Scaling types: Target Tracking (maintain a metric like CPU=50%), Step Scaling (scale by steps based on thresholds), Scheduled Scaling (time-based).

Q: What is a placement group?

Controls how EC2 instances are placed on hardware. Cluster: pack instances in same AZ for low latency, high throughput (HPC). Spread: instances across distinct hardware to reduce simultaneous failure risk. Partition: instances in logical partitions (separate racks), good for Hadoop/Kafka/Cassandra.

3. Storage — S3, EBS, EFS, Glacier

Q: What are S3 storage classes?

- **S3 Standard** — frequently accessed data
 - **S3 Standard-IA** — infrequent access, lower cost, retrieval fee
 - **S3 One Zone-IA** — single AZ, cheaper, less resilient
 - **S3 Intelligent-Tiering** — auto-moves objects between tiers
 - **S3 Glacier Instant** — archive, ms retrieval
 - **S3 Glacier Flexible** — minutes to hours retrieval
 - **S3 Glacier Deep Archive** — cheapest, 12-hour retrieval
-

Q: What is S3 versioning?

Keeps multiple versions of an object in the same bucket. Protects against accidental deletion and overwrites. When enabled, deletes create a Delete Marker rather than removing the object. Cannot be disabled once enabled, only suspended.

Q: What is S3 lifecycle policy?

Rules to automatically transition objects between storage classes or expire them after a set number of days. Example: move to IA after 30 days, Glacier after 90 days, delete after 365 days.

Q: What is S3 Cross-Region Replication (CRR)?

Asynchronously replicates objects from a source bucket to a destination bucket in a different region. Requires versioning enabled on both buckets. Used for compliance, latency reduction, and disaster recovery.

Q: What is the difference between EBS and EFS and S3?

EBS (Elastic Block Store): block storage attached to a single EC2 instance (except io1 Multi-Attach), low latency, like a hard drive. EFS (Elastic File System): shared NFS file system, can mount on hundreds of EC2 instances simultaneously, scales automatically. S3: object storage, accessed via HTTP API, virtually unlimited, ideal for unstructured data, backups, static content.

Q: What are EBS volume types?

- **gp3/gp2** — general purpose SSD, boot volumes
 - **io2/io1** — provisioned IOPS SSD, databases requiring high IOPS
 - **st1** — throughput-optimized HDD, big data, log processing
 - **sc1** — cold HDD, cheapest, infrequent access
-

4. Networking — VPC, Route53, CloudFront

Q: What is a VPC?

Virtual Private Cloud is a logically isolated network in AWS where you launch resources. You define the IP range (CIDR), subnets, route tables, internet gateways, NAT gateways, and security rules.

Q: What is the difference between Public and Private subnets?

Public subnet: has a route to the Internet Gateway (0.0.0.0/0 → IGW) and resources have public IPs. Private subnet: no direct route to IGW; uses a NAT Gateway (in a public subnet) for outbound internet access only.

Q: What is a NAT Gateway vs NAT Instance?

NAT Gateway is AWS-managed, highly available per AZ, no need to manage. NAT Instance is an EC2-based older approach, you manage patching, HA, and scaling. NAT Gateway is always preferred in production.

Q: What is VPC Peering?

A network connection between two VPCs allowing private IP traffic. Non-transitive — if A peers B and B peers C, A cannot talk to C through B. Works across accounts and regions.

Q: What is AWS Transit Gateway?

A hub-and-spoke network transit hub connecting VPCs and on-premises networks. Solves the transitive routing limitation of VPC Peering. Supports thousands of VPCs centrally.

Q: What is a VPC Endpoint?

Allows private connectivity from your VPC to AWS services (like S3, DynamoDB) without using the internet, NAT, or VPN. Interface Endpoints use ENIs and PrivateLink. Gateway Endpoints (free) are only for S3 and DynamoDB via route table.

Q: What is Route 53?

AWS's DNS service. Features: domain registration, DNS routing, health checks. Routing policies: Simple, Weighted (A/B traffic split), Latency-based, Failover (active-passive), Geolocation, Geoproximity, Multi-value Answer.

Q: What is CloudFront?

AWS's CDN that caches content at 450+ edge locations globally. Reduces latency for static and dynamic content. Integrates with S3, ALB, EC2. Supports HTTPS, signed URLs/cookies for private content, Lambda@Edge for edge logic.

5. Load Balancers & ELB

Q: What are the types of AWS Load Balancers?

- **ALB (Application Load Balancer)** — Layer 7, HTTP/HTTPS, path-based and host-based routing, supports WebSockets, best for microservices
- **NLB (Network Load Balancer)** — Layer 4, TCP/UDP, ultra-low latency, millions of requests/sec, static IP support
- **GWLB (Gateway Load Balancer)** — Layer 3, for deploying third-party virtual appliances (firewalls, IDS)

- **CLB (Classic)** — legacy, avoid for new designs
-

Q: What is the difference between ALB and NLB?

ALB operates at HTTP level, can inspect headers/paths/host for routing, supports sticky sessions via cookies. NLB operates at TCP/IP level, cannot inspect HTTP, but handles extreme throughput and preserves client source IP natively. NLB has static IPs per AZ; ALB does not.

Q: What is a Target Group?

A logical grouping of targets (EC2 instances, IPs, Lambda functions, containers) that a load balancer routes traffic to. Health checks are configured at the target group level.

6. IAM

Q: What are IAM components?

Users (individuals), Groups (collection of users), Roles (assumed by services/users, no long-term credentials), Policies (JSON documents defining permissions). Best practice: grant least privilege, avoid root account, use roles for EC2/Lambda, enable MFA.

Q: What is the difference between IAM Role and IAM User?

IAM User has permanent credentials (access key + secret). IAM Role has temporary credentials, assumed by AWS services, applications, or federated users. Roles are preferred for EC2, Lambda, cross-account access, and SSO federation.

Q: What is an IAM Policy?

JSON document with Effect (Allow/Deny), Action (e.g. s3:GetObject), Resource (ARN), and optionally Condition. Types: Identity-based (attached to user/group/role), Resource-based (attached to resource like S3 bucket policy), Permission Boundaries (max permissions a role/user can have), SCPs (at org level).

Q: What is the difference between an IAM policy and an S3 bucket policy?

IAM policy is attached to an identity and says what that identity can do. Bucket policy is attached to the resource and says who can access it. Both can work together — access is

granted if either allows and neither denies.

Q: What is STS (Security Token Service)?

AWS STS generates temporary, short-lived credentials for IAM roles. Used for cross-account access (AssumeRole), federation (AssumeRoleWithWebIdentity, AssumeRoleWithSAML), and EC2 instance profiles.

7. RDS & Databases

Q: What is RDS and which engines does it support?

Relational Database Service is a managed DB service handling provisioning, patching, backups, and replication. Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora.

Q: What is Aurora?

AWS's own MySQL/PostgreSQL-compatible relational DB. 5x faster than MySQL, 3x faster than PostgreSQL. Storage auto-scales to 128 TB. Data replicated 6 ways across 3 AZs. Supports Aurora Serverless (auto-pause and scale on demand).

Q: What is RDS Multi-AZ vs Read Replicas?

Multi-AZ: synchronous replication to standby in another AZ, automatic failover, for high availability. Read Replicas: asynchronous replication, used for read scaling, can be in different regions. Multi-AZ is for HA, Read Replica is for performance.

Q: What is DynamoDB?

AWS's fully managed NoSQL key-value and document database. Single-digit millisecond latency at any scale. Supports DAX (in-memory cache), DynamoDB Streams, Global Tables (multi-region active-active), TTL, and on-demand or provisioned capacity modes.

Q: What is ElastiCache?

Managed in-memory caching service. Supports Redis (persistence, pub/sub, sorted sets, cluster mode) and Memcached (simpler, multi-threaded). Used for session storage, DB query caching, leaderboards, and rate limiting.

8. Lambda & Serverless

Q: What is AWS Lambda?

Serverless compute that runs your code in response to events without managing servers. You pay per invocation and duration (ms). Supports Node.js, Python, Java, Go, Ruby, .NET. Max timeout 15 minutes, max memory 10 GB.

Q: What are Lambda triggers?

API Gateway (HTTP), S3 (object events), DynamoDB Streams, Kinesis, SQS, SNS, EventBridge (scheduled/event rules), Cognito, ALB, and more.

Q: What is a cold start in Lambda?

When a new Lambda execution environment is initialized for the first time (or after inactivity), there is a latency overhead called a cold start. Java/Spring has higher cold starts than Node/Python. Mitigations: Provisioned Concurrency (keeps environments warm), reducing package size, using SnapStart (for Java).

Q: What is Lambda Concurrency?

Number of simultaneous executions. Default account limit: 1000 concurrent executions per region. Reserved Concurrency: guarantees a set number of concurrent executions for one function (also caps it). Provisioned Concurrency: pre-initializes environments to eliminate cold starts.

Q: What is the difference between SQS and SNS?

SQS (Simple Queue Service): pull-based, message queue, one consumer processes each message, decouples producers and consumers, supports Standard (at-least-once) and FIFO (exactly-once, ordered). SNS (Simple Notification Service): push-based, pub/sub, one message fans out to multiple subscribers (SQS, Lambda, HTTP, email). Often used together: SNS → SQS fanout pattern.

9. ECS, EKS & Containers

Q: What is ECS?

Elastic Container Service is AWS's container orchestration service. Runs Docker containers on EC2 (you manage the cluster) or Fargate (serverless, AWS manages infrastructure). Uses

Task Definitions (container config), Tasks (running instance), Services (maintains desired count).

Q: What is Fargate?

Serverless compute engine for containers (works with ECS and EKS). You specify CPU and memory, AWS provisions and manages the underlying EC2 instances. No need to manage or patch servers.

Q: ECS vs EKS?

ECS is AWS-native, simpler, tightly integrated with AWS services, proprietary. EKS is managed Kubernetes, portable (standard K8s API), more complex, better for teams already using K8s or needing multi-cloud portability.

Q: What is ECR?

Elastic Container Registry is AWS's fully managed Docker container image registry. Integrates with ECS, EKS, and Lambda. Supports image scanning for vulnerabilities, lifecycle policies to clean old images, and cross-region replication.

10. Monitoring & Observability

Q: What is CloudWatch?

AWS's monitoring and observability service. Collects metrics (CPU, memory with agent, custom metrics), logs (CloudWatch Logs), events (EventBridge), and supports alarms that trigger actions (SNS, Auto Scaling, Lambda). CloudWatch Agent needed for OS-level metrics.

Q: What is CloudWatch Logs Insights?

Interactive query service to search and analyze log data in CloudWatch Logs in real time. Supports a SQL-like query language. Used for debugging, security analysis, and operational insights.

Q: What is AWS X-Ray?

Distributed tracing service. Instruments your application to trace requests end-to-end across services. Shows latency, errors, service map, and helps identify bottlenecks in microservices and Lambda functions.

Q: What is CloudTrail?

Records all API calls made in your AWS account (who did what, when, from where). Provides governance, compliance, and audit trails. Delivers logs to S3. CloudTrail Insights detects unusual API activity.

Q: What is AWS Config?

Continuously monitors and records AWS resource configurations and evaluates them against desired rules. Enables compliance auditing, change tracking, and configuration history. Example rule: "All S3 buckets must have encryption enabled."

11. Security Services

Q: What is AWS WAF?

Web Application Firewall that protects web applications from common exploits (SQL injection, XSS, bad bots). Integrates with CloudFront, ALB, API Gateway. Uses Web ACLs with rules and rule groups.

Q: What is AWS Shield?

DDoS protection service. Shield Standard (free, always-on, protects against layer 3/4 attacks). Shield Advanced (paid, protects against larger sophisticated attacks, provides DDoS cost protection, 24/7 DDoS response team).

Q: What is AWS GuardDuty?

Intelligent threat detection service that analyzes CloudTrail, VPC Flow Logs, and DNS logs using ML to detect malicious activity (compromised instances, unusual API calls, cryptocurrency mining, data exfiltration).

Q: What is AWS Secrets Manager?

Manages, rotates, and retrieves secrets (DB passwords, API keys). Supports automatic rotation using Lambda. Integrates with RDS, Redshift, DocumentDB. Better than hardcoding secrets or using SSM Parameter Store for sensitive credentials.

Q: What is KMS?

Key Management Service manages cryptographic keys for data encryption. Integrates with most AWS services (S3, EBS, RDS). Supports customer-managed keys (CMK), AWS-managed keys, and CloudHSM for dedicated hardware key storage.

12. API Gateway

Q: What is API Gateway?

Fully managed service to create, publish, secure, and monitor REST, HTTP, and WebSocket APIs. Integrates with Lambda, EC2, HTTP backends. Features: request/response transformation, throttling, caching, API keys, usage plans, authentication (Cognito, Lambda authorizers).

Q: What is the difference between REST API and HTTP API in API Gateway?

HTTP API is cheaper, lower latency, simpler. REST API has more features: usage plans, API keys, request validation, AWS WAF integration, edge-optimized deployment, X-Ray tracing. Use HTTP API for simple Lambda/HTTP proxy; REST API for full feature needs.

13. Infrastructure as Code

Q: What is CloudFormation?

AWS's IaC service that provisions AWS resources using JSON or YAML templates. Manages resources as stacks. Supports change sets (preview changes before applying), drift detection (find manually changed resources), and nested stacks.

Q: What is CDK?

AWS Cloud Development Kit allows defining cloud infrastructure using real programming languages (TypeScript, Python, Java, Go). CDK synthesizes to CloudFormation templates. Offers higher-level constructs (L2/L3) that bundle best practices.

Q: CloudFormation vs Terraform?

CloudFormation: AWS-only, native, tight integration, no state file to manage. Terraform: multi-cloud (AWS, GCP, Azure), uses HCL, maintains a state file, large community module ecosystem. Terraform is preferred for multi-cloud; CloudFormation for AWS-only with deep native integration.

14. Messaging & Streaming

Q: What is Kinesis?

Real-time data streaming service. Kinesis Data Streams: capture and process streaming data in real time (sub-second latency), data retained up to 365 days, multiple consumers via shard. Kinesis Data Firehose: load streaming data to S3, Redshift, OpenSearch automatically. Kinesis Data Analytics: run SQL or Apache Flink on streams.

Q: What is EventBridge?

Serverless event bus for connecting AWS services, SaaS apps, and custom apps. Routes events based on rules to targets (Lambda, SQS, Step Functions, etc.). Supports event replay, schema registry, and cross-account event sharing.

Q: What is Step Functions?

Serverless workflow orchestration service. Defines workflows as state machines (JSON-based Amazon States Language). Supports sequential, parallel, branching, retry, and error handling. Express Workflows for high-volume short-duration; Standard for long-running.

15. Cost Optimization

Q: What tools does AWS provide for cost management?

AWS Cost Explorer (visualize and analyze costs/usage), AWS Budgets (set alerts when costs exceed thresholds), Cost Allocation Tags (tag resources for cost attribution), Trusted Advisor (cost optimization checks), Compute Optimizer (right-sizing recommendations), S3 Storage Lens.

Q: What are common cost optimization strategies?

Right-size EC2 instances using Compute Optimizer, use Reserved Instances or Savings Plans for stable workloads, use Spot for stateless/fault-tolerant workloads, delete

unattached EBS volumes, clean up unused snapshots/AMIs, use S3 Intelligent-Tiering, set S3 lifecycle policies, use Lambda instead of always-on EC2 for bursty workloads.

16. High Availability & Disaster Recovery

Q: What are the DR strategies (RTO/RPO)?

- **Backup & Restore** — highest RTO/RPO, cheapest; restore from backups
 - **Pilot Light** — minimal version of environment always running (core DB replicated), scale up on disaster
 - **Warm Standby** — scaled-down but fully functional environment always running, scale out on disaster
 - **Multi-Site Active-Active** — full production in 2+ regions simultaneously, lowest RTO/RPO, most expensive
-

Q: What is RPO and RTO?

RPO (Recovery Point Objective): maximum acceptable data loss measured in time. How old can the restored data be? RTO (Recovery Time Objective): maximum acceptable downtime. How quickly must the system recover?

Q: How do you achieve HA in AWS?

Multi-AZ deployments, Auto Scaling Groups, ALB across AZs, RDS Multi-AZ, ElastiCache Multi-AZ, S3 (11 9s durability), Route 53 health checks with failover routing, EFS (multi-AZ by default).

AWS Production Issues Q&A

Issue 1: EC2 Instance Unreachable / SSH Fails

What happened: Instance not responding to SSH or ping. Application down.

How to investigate:

- Check instance state in EC2 console (running vs stopped)
- Check System Status Check and Instance Status Check

- Review Security Group — port 22 (SSH) or 443 open from your IP?
- Check NACL — inbound AND outbound rules (stateless)
- Check route table — is there a route to IGW for public subnet?
- Check if Elastic IP attached (if needed)
- Use EC2 Serial Console or EC2 Instance Connect for access without SSH

How to fix:

- If Security Group missing rule → add inbound SSH/RDP rule
- If NACL blocking → add allow rule for port 22 inbound and ephemeral ports outbound
- If instance hung → stop and start (moves to new host; solves hardware issues)
- If disk full → attach another EBS volume, mount, clean up logs
- Use SSM Session Manager as SSH alternative (no open ports needed)

Prevention:

- Always allow SSM Session Manager as a keyless access fallback
- Never open port 22 to 0.0.0.0/0 — restrict to bastion or VPN CIDR
- Enable CloudWatch agent to alert on disk usage > 80%
- Set up status check alarms to auto-recover instance

Issue 2: RDS / Aurora — High CPU or Slow Queries

What happened: Application response times spike, DB CPU at 90%+, connection timeouts.

How to investigate:

- CloudWatch metrics: CPUUtilization, DatabaseConnections, ReadLatency, WriteLatency, FreeableMemory
- RDS Performance Insights: top SQL queries, waits, hosts
- Enable slow query log (MySQL: `slow_query_log=1`, `long_query_time=1`)
- Check connection count — are you exhausting max connections?

How to fix:

- Kill long-running queries immediately via `SHOW PROCESSLIST` and `KILL <id>`
- Add missing indexes identified by slow query log or EXPLAIN plan
- Use Read Replicas to offload read-heavy traffic
- Enable RDS Proxy to pool connections and prevent exhaustion
- Scale instance vertically (change instance class) for immediate relief
- Enable ElastiCache for frequently-read, rarely-changed data

Prevention:

- Set CloudWatch alarms on CPUUtilization > 70% and DatabaseConnections > 80% of max
- Use RDS Proxy in production for all connection-heavy workloads (Lambda especially)
- Review query plans in dev/staging before deploying
- Schedule ANALYZE/OPTIMIZE during low-traffic windows

Issue 3: S3 — AccessDenied Errors

What happened: Application or Lambda getting 403 AccessDenied when reading/writing S3.

How to investigate:

- Check the IAM role attached to the service (EC2 instance profile, Lambda execution role)
- Check the role's IAM policy — does it have `s3:GetObject` / `s3:PutObject` on the correct bucket ARN?
- Check S3 bucket policy — is there an explicit Deny overriding the IAM allow?
- Check S3 Block Public Access settings if public access is needed
- Check KMS key policy if bucket is SSE-KMS encrypted — does the role have `kms:Decrypt` / `kms:GenerateDataKey`?
- Use IAM Policy Simulator to test permissions

How to fix:

- Add missing S3 actions to the IAM role policy
- Remove conflicting Deny statement in bucket policy
- Add `kms:Decrypt` and `kms:GenerateDataKey` to role if bucket uses CMK
- For cross-account: add both the bucket policy (resource-based) and IAM policy (identity-based)

Prevention:

- Use least-privilege IAM policies with specific bucket ARNs (`arn:aws:s3:::my-bucket/*`)
- Test IAM permissions with Policy Simulator before deploying
- Enable S3 server access logging and CloudTrail data events for audit

Issue 4: Lambda Timeouts / Out of Memory

What happened: Lambda function timing out or hitting memory limit, causing invocation failures.

How to investigate:

- CloudWatch Logs: look for `Task timed out after X seconds` or `Runtime exited with error: signal: killed` (OOM)
- Check `Duration`, `Errors`, `Throttles` metrics in CloudWatch
- Check if downstream service (RDS, external API) is slow — the Lambda waits and hits timeout
- Check if connecting to RDS without RDS Proxy (connection setup overhead at scale)

How to fix:

- Increase timeout (max 15 min) and memory allocation (more memory = more CPU too)
- For DB-connected Lambdas: use RDS Proxy to reuse connections
- Add circuit breaker / shorter timeout on downstream HTTP calls
- Move long-running processing to ECS/Fargate or Step Functions
- Use Lambda SnapStart (Java) or optimize initialization code to reduce cold start

Prevention:

- Set memory and timeout based on P99 observed duration, not guesses
 - Set CloudWatch alarm on **Errors** and **Duration** approaching limit
 - Never connect Lambda directly to RDS at scale — always use RDS Proxy
 - Use X-Ray tracing to identify where time is being spent
-

Issue 5: Auto Scaling Not Scaling Out Fast Enough

What happened: Traffic spike hit, ASG didn't scale fast enough, instances were overwhelmed, some requests failed.

How to investigate:

- Check ASG Activity History for scale-out events and timing
- Check CloudWatch metric that triggers scaling and when alarm entered ALARM state
- Check cooldown period (default 300 seconds between scale actions)
- Check if AMI bake time or UserData script is slow — instance takes 5+ min to be healthy
- Check ALB target group health checks — are new instances passing quickly?

How to fix:

- Switch from Simple/Step scaling to Target Tracking (reacts faster)
- Reduce cooldown period
- Use predictive scaling for known traffic patterns
- Optimize UserData / bake dependencies into AMI to reduce launch time
- Reduce ALB health check interval and healthy threshold count

Prevention:

- Use pre-baked AMIs with application already installed — only pull config at launch
- Set aggressive health check on ALB: interval 10s, threshold 2
- Use Scheduled Scaling for known peak times (e.g. weekday 9 AM)

- Always test scaling behavior in load test before production
-

Issue 6: ECS / Fargate Task Keeps Restarting

What happened: ECS service tasks cycling continuously, service never reaches stable state.

How to investigate:

- ECS service events tab — reason for task stopping
- CloudWatch Logs for the container — application crash logs
- Check exit code: exit code 1 = app crash, exit code 137 = OOM killed, exit code 143 = graceful shutdown
- Check task definition CPU/memory — is the limit too low?
- Check health check on target group — is the app healthy before health check fires?

How to fix:

- Fix application crash (check logs for stack trace)
- Increase memory/CPU in task definition for OOM issues
- Add `startPeriod` to container health check to give app time to start before checks begin
- If image pull fails: check ECR permissions on task execution role, check image tag exists

Prevention:

- Set appropriate `startPeriod` (e.g. 60-120s for Spring Boot) in health check
 - Always tag Docker images with version, not just `latest`
 - Set CloudWatch alarm on ECS `service_running_task_count` below desired
 - Test container locally with same memory limits before deploying
-

Issue 7: API Gateway 5XX Errors / Throttling

What happened: Clients receiving 502 Bad Gateway or 429 Too Many Requests from API Gateway.

How to investigate:

- CloudWatch metrics: `5XXError`, `4XXError`, `IntegrationLatency`, `Latency`, `Count`
- 502: integration error — Lambda threw unhandled exception, or response format wrong (missing `statusCode`)
- 504: integration timeout — Lambda ran longer than API Gateway's 29-second limit
- 429: throttling — exceeded account-level (10,000 RPS) or stage-level limit

How to fix:

- 502: fix Lambda function error; ensure response is `{ statusCode, headers, body }` format
- 504: optimize Lambda, move to async pattern (SQS + Lambda), or use async API pattern with polling
- 429: request limit increase via AWS Support, or implement Usage Plans per client API key
- Enable API Gateway caching (TTL-based) for GET endpoints to reduce Lambda invocations

Prevention:

- Set usage plans and throttling per client to prevent one consumer starving others
- Use Lambda destinations or SQS for long-running async operations instead of sync API Gateway → Lambda
- Monitor `IntegrationLatency` — alert if P99 > 5 seconds
- Return proper structured errors from Lambda to avoid 502s

Issue 8: CloudFront Serving Stale Content

What happened: Deployed new version of static assets but users still seeing old version.

How to investigate:

- Check CloudFront `Cache-Control` and `Expires` headers on origin response

- Check CloudFront cache behavior — is it set to cache based on query string, headers, cookies?
- Check TTL settings: min, default, max TTL in cache behavior
- Use `curl -I <cloudfront-url>` and check `X-Cache: Hit from cloudfront` vs `Miss`

How to fix:

- Invalidate specific paths: `aws cloudfront create-invalidation --distribution-id <id> --paths "/static/*"`
- For releases: use versioned file names (e.g. `main.abc123.js`) — new file name = cache miss automatically, no invalidation needed

Prevention:

- Always use content-hash-based file names for JS/CSS (Webpack/Vite does this by default)
- Reserve invalidations for HTML files only (they reference the hashed assets)
- Set short TTL (60-300s) on HTML, long TTL (1 year) on hashed static assets
- Note: CloudFront invalidations take ~1 min and cost after 1,000/month free

Issue 9: DynamoDB ProvisionedThroughputExceededException

What happened: Application getting throttling errors from DynamoDB under load.

How to investigate:

- CloudWatch metrics: `ConsumedReadCapacityUnits`, `ConsumedWriteCapacityUnits`, `ThrottledRequests`
- Check if using provisioned mode with capacity too low
- Check for hot partition — one partition key receiving disproportionate traffic

How to fix:

- Switch to On-Demand capacity mode for unpredictable workloads (immediate fix, no capacity planning)
- If staying on provisioned: increase RCU/WCU or enable auto-scaling

- Hot partition: redesign partition key — add random suffix, use write sharding, or use composite keys to distribute load
- Enable DAX (DynamoDB Accelerator) for read-heavy workloads to absorb read traffic

Prevention:

- Design partition keys for uniform distribution from day one
- Use on-demand mode for new tables until traffic patterns are understood
- Set CloudWatch alarms on `ThrottledRequests > 0`
- Review access patterns before table design — DynamoDB is access-pattern-driven

Issue 10: VPC Network Connectivity Failure

What happened: Service A cannot reach Service B across subnets or VPCs. Connection refused or timed out.

How to investigate:

- Timed out = Security Group or NACL blocking (packet dropped)
- Connection refused = port reached but nothing listening (app not running, wrong port)
- Use VPC Reachability Analyzer — point-to-point network path analysis, identifies which rule is blocking
- Check Security Group of target — is source's SG or CIDR in inbound rules?
- Check NACL of target subnet — inbound rule AND outbound ephemeral port rule (1024-65535)?
- Check route table — is there a route between subnets or across VPCs (peering, TGW)?

How to fix:

- Add missing Security Group inbound rule referencing source SG (not IP, so it's dynamic)
- Add NACL rules for both inbound traffic port AND outbound ephemeral ports
- Add VPC peering route to route table if cross-VPC
- Check TGW route tables if using Transit Gateway

Prevention:

- Reference Security Groups by ID in rules (not CIDR) for dynamic EC2 environments
 - Use VPC Reachability Analyzer as part of deployment validation
 - Document and version control Security Group rules in Terraform/CloudFormation
 - Enable VPC Flow Logs → CloudWatch to diagnose rejected traffic patterns
-

Issue 11: Billing Spike — Unexpected AWS Cost Increase

What happened: Monthly AWS bill 3x higher than expected.

How to investigate:

- AWS Cost Explorer: filter by service, region, usage type to find the spike
- Check for NAT Gateway data processing charges (common culprit — all outbound from private subnet charged per GB)
- Check for data transfer charges between AZs or regions
- Check for accidentally left-on resources: EC2, RDS, NAT Gateway, Elastic IPs not attached
- Check CloudTrail for unusual API activity (compromised credentials spinning up resources)

How to fix:

- Terminate unused resources immediately
- For NAT Gateway cost: route S3/DynamoDB traffic through VPC Gateway Endpoints (free) instead of NAT
- For inter-AZ traffic: co-locate services in same AZ where latency allows
- If compromised: rotate credentials, revoke active sessions via IAM, audit CloudTrail

Prevention:

- Set AWS Budgets alerts at 80% and 100% of expected monthly spend
- Enable Cost Anomaly Detection (ML-based, alerts on unusual spend patterns)
- Tag all resources and use cost allocation tags for per-team visibility

- Regular Trusted Advisor review for idle/unattached resources
 - Enable GuardDuty to detect credential compromise early
-

Issue 12: SQS Messages Not Being Processed / Building Up

What happened: SQS queue depth growing, messages not being consumed, Lambda or consumer not processing.

How to investigate:

- CloudWatch: `ApproximateNumberOfMessagesVisible` growing, `NumberOfMessagesSent` vs `NumberOfMessagesDeleted`
- If Lambda consumer: check Lambda `Errors` and `Throttles` metrics
- Check if messages are in the Dead Letter Queue (DLQ) — indicates processing failures
- Check Lambda concurrency limits — is it throttled?
- Check SQS visibility timeout vs Lambda timeout — if visibility timeout < Lambda execution time, message becomes visible again and is processed twice

How to fix:

- Fix the bug causing Lambda failures (check DLQ messages for clues)
- Set SQS visibility timeout to 6x Lambda timeout (AWS recommendation)
- Increase Lambda reserved concurrency or request account limit increase
- Process DLQ messages manually after fixing the bug

Prevention:

- Always configure a DLQ on SQS queues — never lose failed messages
 - Set CloudWatch alarm on DLQ `ApproximateNumberOfMessagesVisible > 0`
 - Set visibility timeout = 6 × Lambda timeout
 - Use SQS FIFO only when ordering is truly required — Standard is faster and cheaper
-

That covers the full AWS interview Q&A plus production issues with diagnosis, fix, and prevention for every scenario. Let me know if you want any section expanded or more scenarios added.

6/5/2026, 10:28:17 AM



give at least 10 each sections production issues on EC2, s3, RDS, Lamda, ASG, ECS, API gateway & SQS (similar way, issue -> fix & analysis -> prevention)

6/5/2026, 10:32:45 AM



AWS Production Issues — Deep Dive

EC2 Production Issues

Issue 1: EC2 Instance Fails Status Checks

What happened: Instance shows "1/2 checks failed" or "2/2 checks failed" in console. Application unreachable.

System Status Check failure = AWS hardware/host problem. Instance Status Check failure = OS/software problem.

How to investigate:

- EC2 Console → Instances → Status Checks tab
- System check fail: AWS infrastructure issue, not your fault
- Instance check fail: OS crashed, disk full, kernel panic, misconfigured network interface
- Check CloudWatch metrics: CPUUtilization, StatusCheckFailed_System, StatusCheckFailed_Instance
- Try EC2 Serial Console for direct OS-level access without SSH

How to fix:

- System check fail → Stop and Start (migrates to new host). Do NOT just reboot — reboot stays on same host
- Instance check fail → Serial Console to diagnose, or detach root EBS, attach to another instance, repair filesystem, reattach

- If disk full: attach a second EBS volume, mount, delete large log files, expand root volume
- Enable Auto Recovery: CloudWatch alarm on `StatusCheckFailed_System` → EC2 recover action

Prevention:

- Set CloudWatch alarm: `StatusCheckFailed_Instance` > 0 → SNS alert
- Set CloudWatch alarm: `StatusCheckFailed_System` > 0 → EC2 Auto Recover action
- Monitor disk usage via CloudWatch Agent — alert at 80%, critical at 90%
- Use log rotation (logrotate) for all application logs
- Run instances in ASG so failed instances are replaced automatically

Issue 2: EC2 High CPU — Application Unresponsive

What happened: CPU pegged at 95-100%, application response times explode or instance stops responding to health checks.

How to investigate:

- CloudWatch: `CPUUtilization` metric — when did it start? Correlates with a deployment or traffic event?
- SSH in and run `top` or `htop` — which process is consuming CPU?
- `ps aux --sort=-%cpu | head -20` for top CPU consumers
- Check for runaway application thread, stuck GC (Java), cron job, or crypto mining (if compromised)
- Check memory: `free -h` — are you swapping? Swap causes high CPU due to thrashing
- For Java: `jstack <pid>` to get thread dump, identify blocked/spinning threads

How to fix:

- Kill the offending process if runaway: `kill -9 <pid>`
- For Java GC issues: restart app, then investigate heap sizing

- Increase instance type temporarily (stop → change instance type → start)
- If it's a traffic spike: add to ASG, scale out horizontally
- If compromised (crypto miner): isolate instance (remove from SG/ALB), take snapshot, terminate, investigate CloudTrail

Prevention:

- CloudWatch alarm: CPUUtilization > 80% for 5 minutes → SNS + Auto Scaling trigger
- Use T-series instances only for dev — in production use M or C series without burst credits
- Profile application in load testing before production
- Enable GuardDuty to detect crypto mining signatures
- Run instances in ASG — unhealthy instances replaced automatically

Issue 3: EC2 Disk Full — Application Crashes

What happened: Application writes fail, database errors, OS cannot write temp files.
Instance may become unresponsive.

How to investigate:

- `df -h` — which filesystem is full?
- `du -sh /*` → drill down to find largest directories
- Common culprits: application logs in /var/log, core dumps in /tmp, Docker images in /var/lib/docker, old kernel packages
- `lsdf | grep deleted` — files deleted but still held open by a process (don't free space until process restarted)

How to fix:

- Immediate: delete old log files, clear /tmp, remove unused Docker images (`docker system prune`)
- Expand EBS volume: Elastic Volumes allows resizing without downtime. Resize volume in console → run `growpart` and `resize2fs` inside OS
- Attach a new EBS volume for logs or data and symlink/mount to the full path

- Restart process holding deleted files open to free inode space

Prevention:

- CloudWatch Agent: monitor disk usage, alarm at 80%
- Configure logrotate for all application logs with daily rotation and 7-day retention
- Use CloudWatch Logs agent to stream logs off the instance — logs don't accumulate locally
- Set EBS volume size generously or enable auto-expanding EBS with Lambda + CloudWatch alarm
- Never use root volume for application data — mount a separate EBS for /data or /logs

Issue 4: EC2 Memory Exhaustion / OOM Killer

What happened: Processes randomly killed, application crashes without obvious errors.

`dmesg` shows "Out of memory: Kill process".

How to investigate:

- `dmesg | grep -i "oom\|kill"` — confirms OOM killer fired
- `free -h` — current memory state
- CloudWatch does NOT show memory by default — need CloudWatch Agent
- Check which process was killed in dmesg — it's usually your app or a critical system process
- For Java: check heap size vs available RAM. Is Xmx too close to total RAM? No room for OS + off-heap

How to fix:

- Immediate: restart killed process
- If Java: reduce Xmx or increase instance size
- Add swap space as a safety net (not a fix): `dd if=/dev/zero of=/swapfile bs=1G count=4 && mkswap /swapfile && swapon /swapfile`
- Move to larger instance type (memory-optimized if needed: r6i, x2i)

Prevention:

- Install CloudWatch Agent to publish MemoryUtilization metric — alert at 80%
 - Set Java Xmx to 75% of instance RAM maximum — leave headroom for OS and off-heap
 - Enable swap as a safety net on all instances
 - Profile memory usage under load in staging before production
 - Use memory-optimized instances (r6i) for Java microservices with large heaps
-

Issue 5: EC2 Instance Unreachable After Security Group Change

What happened: Someone modified a Security Group rule, locked out all SSH/HTTPS access.

How to investigate:

- EC2 Console → Security Groups → check inbound rules — is your IP or CIDR missing?
- CloudTrail → filter by eventName=AuthorizeSecurityGroupIngress / RevokeSecurityGroupIngress — find who changed what and when
- VPC Reachability Analyzer — confirm path is blocked at SG level

How to fix:

- Re-add the correct inbound rule (port 22 from bastion CIDR, or 443 from ALB SG)
- If completely locked out of SSH: use SSM Session Manager (no port 22 needed), EC2 Instance Connect, or Serial Console
- If SSM agent not installed: stop instance, detach root EBS, mount to a recovery instance, edit /etc/ssh/sshd_config or authorized_keys, reattach

Prevention:

- Never rely on direct SSH from the internet — use SSM Session Manager as the standard access method
- Restrict SG changes via IAM: deny ec2:RevokeSecurityGroupIngress for production SGs
- Use AWS Config rule: restricted-ssh — alerts if port 22 is open to 0.0.0.0/0

- Require MFA for console access and for destructive IAM actions
-

Issue 6: EC2 Instance Not Joining Auto Scaling Group Properly

What happened: New instance launches, passes EC2 status checks, but ASG marks it unhealthy and terminates it.

How to investigate:

- ASG Activity History tab — termination reason shown (e.g. "ELB instance failed to respond to a health check")
- ALB Target Group → Targets — is the instance showing unhealthy? What is the health check response?
- SSH into a new instance and manually hit the health check endpoint: `curl http://localhost:8080/health`
- Check application startup logs — is the app crashing on startup?
- Is the health check path correct? Is the app starting slow (Java cold start) and failing checks before it's ready?

How to fix:

- Increase health check grace period in ASG (time after launch before health checks begin) — for Spring Boot set 120-180s
- Fix the application startup failure visible in logs
- Correct the health check path in the ALB target group
- Reduce health check interval and threshold count so healthy instances are recognized faster

Prevention:

- Set ASG health check grace period = application startup time + 30s buffer
- Use a dedicated lightweight `/health` endpoint (Spring Actuator) that responds in <100ms
- Bake application into AMI — reduces startup time from 3 min to 30 sec
- Test UserData scripts manually on an EC2 before baking into launch template

Issue 7: EC2 Network Performance Degradation

What happened: Network throughput drops, latency spikes between EC2 instances or to RDS.

How to investigate:

- `iperf3` between instances to measure actual throughput
- CloudWatch: NetworkIn, NetworkOut, NetworkPacketsIn metrics
- Check instance type network baseline — t3.micro has burst networking, not dedicated; c5n has up to 100 Gbps
- Check if EC2 instance is in a placement group — Cluster placement group needed for low latency HPC
- `ethtool -i eth0` and check for ENA driver — Elastic Network Adapter required for enhanced networking

How to fix:

- Upgrade to instance type with higher network baseline (c5n, m5n for network-intensive)
- Enable Enhanced Networking (ENA) — most modern instance types have this by default
- Use Cluster Placement Group for latency-sensitive workloads
- For egress-heavy apps: use VPC Gateway Endpoint for S3/DynamoDB (free and bypasses NAT)

Prevention:

- Test network throughput in staging with same instance type as production
- Choose instance types with dedicated ENA for production network-heavy workloads
- Use placement groups for any distributed system requiring low inter-node latency (Kafka, Cassandra)
- Monitor NetworkOut — a sudden spike can indicate data exfiltration or misconfigured replication

Issue 8: EC2 Spot Instance Interruption Causing Downtime

What happened: Spot instances in ASG were interrupted by AWS with 2-minute notice. Traffic dropped as instances were terminated.

How to investigate:

- EC2 → Spot Requests → check interruption notices
- CloudWatch Events: AWS emits EC2 Spot Instance Interruption Warning 2 minutes before termination
- Check if instances were draining properly from ALB before termination
- ASG activity history shows rapid terminations across multiple instances simultaneously

How to fix:

- Use mixed instances policy in ASG: combine On-Demand (base capacity) + Spot (scale capacity)
- Use multiple instance types and AZs for Spot to reduce simultaneous interruption probability
- Configure Spot interruption handler (Lambda triggered by EventBridge) to gracefully drain connections

Prevention:

- Never run 100% Spot for stateful or critical services
- Use Spot only for stateless, fault-tolerant, or batch workloads
- Set base On-Demand capacity of at least 20-30% of minimum desired count
- Configure capacity-optimized allocation strategy (AWS picks least-interruption pools)
- Use EventBridge rule: EC2 Spot Instance Interruption Warning → Lambda → deregister from ALB early

Issue 9: EC2 Key Pair Lost — Cannot SSH

What happened: Private key (.pem file) lost or team member who had it left. Cannot SSH into instance.

How to investigate:

- Confirm no other way in: SSM Session Manager? EC2 Instance Connect? Another user's key in `authorized_keys`?
- Check if SSM agent is running: `aws ssm describe-instance-information`

How to fix:

- If SSM Session Manager available: connect via SSM, add new public key to `~/.ssh/authorized_keys`, reconnect via SSH with new key
- If SSM not available: stop instance → detach root EBS → attach to a recovery EC2 → mount → edit `/home/ec2-user/.ssh/authorized_keys` → unmount → reattach → restart original instance
- You cannot recover an EC2 key pair — AWS does not store the private key

Prevention:

- Store key pairs in AWS Secrets Manager or a team password manager (1Password, HashiCorp Vault)
- Mandate SSM Session Manager as the primary access method — no key pairs needed for SSH at all
- Use EC2 Instance Connect for temporary access — injects a one-time key valid for 60 seconds
- Rotate key pairs as part of employee offboarding process

Issue 10: EC2 Instance Time Drift Causing TLS / JWT Failures

What happened: Application starts throwing SSL handshake errors, JWT token validation failures, or Kerberos authentication errors despite valid certificates and tokens.

How to investigate:

- On the instance: `date` — is the time correct?
- `chronyc tracking` or `ntpstat` — is NTP synced?
- TLS errors with wrong time: handshake fails because certificate validity window doesn't match system clock

- JWT "token not yet valid" / "token expired" errors when clock is off by even a few minutes

How to fix:

- Force NTP sync: `chronyc makestep` (Amazon Linux 2) or `timedatectl set-ntp true`
- Restart chronyd: `systemctl restart chronyd`
- Verify: `chronyc tracking` should show System time offset < 1ms

Prevention:

- AWS instances use Amazon Time Sync Service (169.254.169.123) by default — ensure it's not blocked
- Never disable chronyd/NTP service
- CloudWatch Custom Metric: publish NTP offset and alert if > 500ms
- For containers: container time comes from host — ensure ECS/EC2 host NTP is healthy

S3 Production Issues

Issue 1: S3 Objects Accidentally Deleted

What happened: Critical objects deleted from S3 — either by a bug, wrong CLI command, or automated script.

How to investigate:

- Is versioning enabled? If yes, objects have a Delete Marker, not permanently gone
- S3 Console → Show versions → find the Delete Marker for the object
- CloudTrail: filter by eventName=DeleteObject, eventSource=s3.amazonaws.com — who deleted what and when?
- If versioning was not enabled: object is permanently gone unless you have a backup

How to fix:

- Versioning enabled: delete the Delete Marker to restore the object. AWS CLI: `aws s3api delete-object --bucket <bucket> --key <key> --version-id <delete-`

`marker-version-id>`

- If MFA Delete was enabled: you need MFA to remove the Delete Marker (extra protection)
- Versioning not enabled: restore from S3 Replication destination bucket (if CRR was configured), or from backup

Prevention:

- Enable versioning on all production S3 buckets — non-negotiable
- Enable MFA Delete on critical buckets (requires root account to enable)
- Use S3 Object Lock (WORM) for compliance data — cannot be deleted within retention period
- Set S3 bucket policy to deny s3:DeleteObject for all except a specific admin role
- Enable S3 Replication to a second bucket in another account for critical data

Issue 2: S3 Bucket Made Public — Data Exposure

What happened: A bucket or object was made public accidentally exposing sensitive data. Security team or external scanner detected it.

How to investigate:

- S3 Console: Bucket → Permissions → Block Public Access — is it off?
- Check bucket policy for `"Principal": "*" with "Effect": "Allow"`
- Check Object ACLs — were objects uploaded with public-read ACL?
- AWS Config rule: s3-bucket-public-read-prohibited — did it fire?
- GuardDuty finding: Policy:S3/BucketAnonymousAccessGranted

How to fix:

- Immediate: Enable S3 Block Public Access at account level — this overrides all bucket/object ACLs and policies
- Remove offending bucket policy statements with `"Principal": "*" with "Effect": "Allow"`
- Audit all objects in the bucket for public ACLs: `aws s3api list-objects` and check each ACL

- Notify security team, assess what data was exposed and for how long (CloudTrail + S3 access logs)

Prevention:

- Enable S3 Block Public Access at the AWS account level — prevents any bucket from becoming public
- Enable AWS Config rule s3-bucket-public-read-prohibited — auto-remediation to re-block
- Enable S3 Server Access Logging to track who accessed what
- SecurityHub check: S3.2 — S3 buckets should prohibit public read access
- Never grant public access via ACL — use pre-signed URLs for temporary access to private objects

Issue 3: S3 Pre-Signed URL Errors — 403 on Valid URL

What happened: Application generates pre-signed URLs but users get 403 AccessDenied even on fresh URLs.

How to investigate:

- Is the URL already expired? Check expiry parameter in URL (**X-Amz-Expires**)
- Who signed the URL? IAM user or IAM role? If IAM role: the temporary credentials used to sign may have already expired — role session shorter than URL expiry
- Clock skew: generating server's time vs S3 time off by more than 15 minutes → signature invalid
- Was the bucket policy or Block Public Access changed after URL generation?
- Is the signing region correct? URL signed in us-east-1 but bucket is ap-south-1 → mismatch

How to fix:

- Regenerate URL with correct region (**--region ap-south-1** in CLI or set in SDK config)
- If role session expiry is the issue: sign with IAM user credentials, or reduce URL expiry to less than role session duration

- Fix server clock (NTP sync)
- Ensure bucket policy does not have a Deny that overrides the pre-signed URL permission

Prevention:

- Always sign pre-signed URLs using long-lived IAM user or instance profile credentials, not short-lived assumed role tokens unless role session >> URL expiry
- Set URL expiry to the minimum needed (15 min for downloads, seconds for uploads)
- Ensure application servers have NTP synced — signature validation uses timestamp
- Use VPC Endpoint for S3 in private subnets to avoid NAT and reduce latency on uploads/downloads

Issue 4: S3 High Request Costs — Unexpected GET/LIST Charges

What happened: S3 bill shows millions of GET or LIST requests, far more than expected. Cost spiked.

How to investigate:

- S3 Storage Lens: request metrics per bucket, prefix, storage class
- Enable S3 Server Access Logs → query in Athena to find which IPs/user agents are making requests
- Check if CloudFront is NOT in front of S3 — every origin fetch is a GET request charged
- Check if application is doing `ListObjectsV2` in a loop (common bug — listing entire bucket on every request)
- Check if other AWS accounts or public are accessing the bucket

How to fix:

- Put CloudFront in front of S3 for static content — CloudFront caches, dramatically reduces origin GETs
- Fix code doing unnecessary ListObjects — cache the result, use specific key paths instead
- Use S3 Select to retrieve only needed data from large objects instead of full GETs

- Use S3 Inventory (daily/weekly batch file) instead of ListObjects for bulk enumeration

Prevention:

- Always serve static assets through CloudFront — never directly from S3 for public-facing content
- Never do ListObjects in a hot code path — it's slow and expensive
- Enable S3 Storage Lens for ongoing visibility into request patterns
- Set bucket policy to restrict access to known principals only — block unintended consumers

Issue 5: S3 Upload Failing for Large Files

What happened: Application fails to upload files > 100 MB to S3. Timeout errors, incomplete uploads.

How to investigate:

- Check error: `EntityTooLarge` (exceeded 5 GB single PUT limit) or timeout on slow network
- Single PUT limit: 5 GB max. Objects > 5 GB require Multipart Upload
- Check if application is using SDK's managed upload (handles multipart automatically) or raw PUT
- Check for incomplete/aborted multipart uploads accumulating: `aws s3api list-multipart-uploads --bucket <name>`

How to fix:

- Use AWS SDK's managed transfer (TransferManager in Java SDK, managed_upload in Python boto3) — handles multipart automatically for files > threshold
- Set multipart_threshold and multipart_chunksize explicitly in SDK config
- Clean up incomplete multipart uploads: `aws s3api abort-multipart-upload` or set S3 lifecycle rule

Prevention:

- Always use SDK TransferManager/managed upload for any file that could be > 5 MB
 - Set S3 lifecycle rule: abort incomplete multipart uploads after 7 days — prevents storage cost accumulation from abandoned uploads
 - Set server-side timeout generously for upload endpoints
 - For very large files (10s of GB): use S3 Transfer Acceleration for global users
-

Issue 6: S3 Cross-Region Replication Not Working

What happened: Objects added to source bucket are not appearing in destination bucket. CRR configured but not replicating.

How to investigate:

- S3 Console → Replication → check replication rule status — is it enabled?
- S3 Replication metrics: ReplicationLatency, BytesPendingReplication in CloudWatch
- Was versioning enabled on BOTH buckets before CRR was set up? Required on both
- Check the IAM role used by replication — does it have s3:GetObject on source and s3:ReplicateObject on destination?
- CRR only replicates objects added AFTER replication was enabled — existing objects are NOT replicated retroactively

How to fix:

- Enable versioning on both buckets if not already done
- Fix IAM role permissions for the replication service principal
- For existing objects: use S3 Batch Replication (explicitly replicate existing objects)
- Check if objects are encrypted with SSE-KMS — replication role needs kms:Decrypt on source CMK and kms:GenerateDataKey on destination CMK

Prevention:

- Enable versioning and CRR at bucket creation time, before any objects are written
- Use S3 Replication Time Control (RTC) if you need guaranteed 15-minute replication SLA
- Monitor ReplicationLatency CloudWatch metric — alert if > 15 minutes

- Test replication by uploading a test object and verifying it appears in destination within 5 minutes
-

Issue 7: S3 Bucket Policy Locking Out All Access

What happened: Modified S3 bucket policy accidentally blocked all access including your own IAM user. `aws s3 ls s3://bucket` returns AccessDenied.

How to investigate:

- CloudTrail: find the PutBucketPolicy event — what was the policy content?
- Check policy for `"Effect": "Deny"` with `"Principal": "*"` and missing Condition — this denies everyone
- Note: S3 bucket policy cannot lock out the AWS account root user (root can always delete the bucket policy)

How to fix:

- Use root account (or an IAM user not covered by the Deny) to update or delete the bucket policy
- AWS CLI: `aws s3api delete-bucket-policy --bucket <name>` — removes all bucket policies, falls back to IAM-only access
- Rewrite the policy carefully: use explicit principal ARNs, test with Policy Simulator before applying

Prevention:

- Test bucket policies in Policy Simulator before applying to production
 - Apply bucket policies via CloudFormation/Terraform — peer review required, not ad-hoc console edits
 - Use IAM conditions to restrict who can call PutBucketPolicy
 - Keep a documented backup of the current bucket policy before every change
-

Issue 8: S3 Event Notification Not Triggering Lambda

What happened: Objects uploaded to S3 but the configured Lambda trigger is not firing.

How to investigate:

- S3 Console → Bucket → Properties → Event Notifications — is the notification still configured?
- Lambda Console → Configuration → Triggers — is S3 shown as a trigger?
- Check Lambda resource-based policy — does S3 have `lambda:InvokeFunction` permission?
- Check if the S3 event filter prefix/suffix matches the actual object key being uploaded (e.g. prefix "uploads/" but object is "Uploads/")
- Check Lambda execution role — does it have necessary permissions for what it's doing?
- Check Lambda CloudWatch logs — is it being invoked but crashing silently?

How to fix:

- Re-add S3 trigger via Lambda console (automatically adds resource-based policy)
- Add resource policy manually: `aws lambda add-permission --function-name <fn> --statement-id s3-invoke --action lambda:InvokeFunction --principal s3.amazonaws.com --source-arn <bucket-arn>`
- Fix prefix/suffix filter to match actual object key pattern (case-sensitive)
- Check dead-letter queue on Lambda for failed async invocations

Prevention:

- Always add S3 → Lambda triggers via Lambda console or IaC — never manually edit resource policies
- Configure Lambda DLQ (SQS or SNS) to capture failed async invocations
- Set CloudWatch alarm on Lambda Errors > 0
- Use EventBridge as an alternative to S3 native events for more filtering flexibility

Issue 9: S3 CORS Errors — Browser Requests Blocked

What happened: Frontend JavaScript application getting CORS errors when fetching assets or uploading to S3 directly.

How to investigate:

- Browser DevTools → Network tab → preflight OPTIONS request failing? Or actual GET/PUT?
- S3 Console → Bucket → Permissions → CORS — is a CORS configuration present?
- Check allowed origins — is the frontend's domain (e.g. <https://app.example.com>) listed?
- Check allowed methods — GET, PUT, POST as needed?
- Is the request going through CloudFront? CloudFront may need to be configured to forward Origin header

How to fix:

- Add or update S3 CORS configuration:

```
[{
  "AllowedHeaders": ["*"],
  "AllowedMethods": ["GET", "PUT", "POST"],
  "AllowedOrigins": ["https://app.example.com"],
  "ExposeHeaders": ["ETag"],
  "MaxAgeSeconds": 3000
}]
```

- For CloudFront: create a cache behavior that forwards the Origin header and whitelist CORS response headers

Prevention:

- Configure CORS at bucket creation via IaC — not ad-hoc
- Never use `"AllowedOrigins": ["*"]` in production for buckets with authenticated content
- Test CORS with `curl -H "Origin: https://app.example.com" -I https://bucket.s3.amazonaws.com/object` before going live

- For presigned URL uploads (PUT), ensure PUT is in AllowedMethods and ETag is in ExposeHeaders
-

Issue 10: S3 Lifecycle Policy Not Transitioning Objects

What happened: Objects not moving to Glacier or being deleted as per lifecycle rules. Storage costs not decreasing as expected.

How to investigate:

- S3 Console → Bucket → Management → Lifecycle Rules — is the rule enabled?
- Check rule scope: is it applied to the entire bucket or a specific prefix? Does it match your objects?
- Lifecycle transitions have minimum size requirements: objects < 128 KB are not transitioned to IA or Glacier (AWS charges would be more than benefit)
- Check if objects are using non-current version transitions vs current version transitions — versioned buckets need separate rules for current and non-current
- Lifecycle rules run once daily — changes take 24 hours to take effect

How to fix:

- Correct the prefix filter if it doesn't match object keys
- Add separate non-current version lifecycle rules for versioned buckets
- For small objects (< 128 KB): use S3 Intelligent-Tiering instead — no minimum size restriction
- Lifecycle transitions are not retroactive to objects created before the rule — they apply going forward

Prevention:

- Test lifecycle rules on a small test bucket with test objects before applying to production
- For versioned buckets: always add rules for both current and non-current versions, and a rule to expire delete markers

- Use S3 Storage Lens to verify storage class distribution is changing as expected after lifecycle rules are applied
- Document expected cost savings and set a reminder to validate in 30 days

RDS Production Issues

Issue 1: RDS Connection Exhaustion — "Too Many Connections"

What happened: Application throws "Too many connections" errors. New DB connections refused. Application partially or fully down.

How to investigate:

- CloudWatch: DatabaseConnections metric — is it at max_connections?
- `SHOW VARIABLES LIKE 'max_connections'` — what is the current limit?
- `SELECT * FROM information_schema.processlist ORDER BY time DESC` — who is holding connections?
- Is Lambda connecting directly to RDS? Lambda can scale to thousands of concurrent executions, each opening a DB connection — connection pool is useless in Lambda
- Are application instances not releasing connections properly? Connection leak?

How to fix:

- Immediate: `KILL <id>` for idle/sleeping connections consuming slots
- Increase max_connections via RDS parameter group (requires reboot for static parameters)
- Deploy RDS Proxy immediately — connection pooler in front of RDS, pins thousands of app connections to a small pool of real DB connections
- Restart application instances to reset leaked connections

Prevention:

- Use RDS Proxy for all Lambda → RDS connections — mandatory, not optional
- Use connection pooling (HikariCP in Spring Boot) for EC2/ECS workloads — configure pool max to (max_connections / instance_count) - 20%

- Set idle connection timeout in HikariCP to reclaim connections from idle threads
- CloudWatch alarm: DatabaseConnections > 80% of max_connections → SNS alert

Issue 2: RDS Failover Took Too Long — Downtime Exceeded SLA

What happened: RDS Multi-AZ failover triggered (AZ failure or instance crash). Application was down for 3-5 minutes instead of expected <1 minute.

How to investigate:

- RDS Events log: check failover start and end timestamps
- Application logs: when did connection errors start and stop?
- Is the application caching the old DB hostname/IP? JDBC connections don't automatically reconnect after failover — they hold the old IP
- Is DNS TTL in the application too high? RDS failover works by updating DNS CNAME — if app caches DNS too long, it keeps hitting the failed primary
- Is the connection pool configured to test and evict dead connections?

How to fix:

- Set JDBC connection URL to use RDS endpoint (CNAME) not the IP
- Configure HikariCP: `connectionTestQuery`, `keepaliveTime`, `maxLifetime` to detect and replace dead connections
- Set JVM DNS TTL: `-Djava.security.properties` → `networkaddress.cache.ttl=10` (default is 60s)
- Configure retry logic in the application layer for transient connection errors

Prevention:

- Always use RDS Cluster/Writer endpoint (Aurora) or RDS CNAME endpoint — never hardcode IP
- Set JVM DNS TTL to 10-30 seconds for AWS resources
- Configure HikariCP with connection health checks: `connectionTestQuery=SELECT 1`, `keepaliveTime=30000`

- Test Multi-AZ failover in staging: `aws rds reboot-db-instance --db-instance-identifier <id> --force-failover`
- Use Aurora instead of RDS Multi-AZ — Aurora failover is typically 30 seconds vs 1-2 minutes for RDS

Issue 3: RDS Storage Full — Database Stopped

What happened: RDS instance stopped accepting writes. Console shows storage at 100%. AWS auto-stopped the DB to prevent corruption.

How to investigate:

- CloudWatch: FreeStorageSpace metric — when did it reach 0?
- RDS Console: instance status "Storage-full"
- Connect to DB: check which databases/tables are growing: `SELECT table_schema, SUM(data_length + index_length) FROM information_schema.tables GROUP BY table_schema`
- Check binary logs: binlogs accumulate if not purged and consume storage
- Check for large temporary tables or uncontrolled table growth

How to fix:

- Modify RDS instance to increase allocated storage — can be done with no downtime (uses Elastic Volumes)
- Purge binary logs: `CALL mysql.rds_rotate_slow_log; CALL mysql.rds_purge_binlogs(now())`
- Drop or truncate large temp/log tables after confirming safety
- Enable RDS Storage Auto-scaling (set a max limit) — AWS grows storage automatically when < 10% free

Prevention:

- Enable RDS Storage Auto-scaling on all production instances with a safe maximum
- CloudWatch alarm: FreeStorageSpace < 20% of total → SNS alert

- Set binlog retention: `CALL mysql.rds_set_configuration('binlog retention hours', 24)` — don't retain more than needed
- Schedule regular `OPTIMIZE TABLE` for tables with high delete rates to reclaim fragmented space
- Review table growth trends monthly via CloudWatch or custom metrics

Issue 4: Slow Queries Degrading Entire Database

What happened: One or two slow queries monopolize DB threads, causing all other queries to queue and time out.

How to investigate:

- Performance Insights: top SQL by load, wait events (CPU, IO, Locks)
- Enable slow query log: `slow_query_log=1`, `long_query_time=1` in parameter group
- `SHOW PROCESSLIST` — are there long-running queries blocking others?
- Run `EXPLAIN` on the slow query — is it doing a full table scan (type=ALL in output)?
- Check for missing indexes, N+1 query patterns, unbounded queries (no LIMIT, full table joins)

How to fix:

- Kill the offending query immediately: `KILL QUERY <id>`
- Add the missing index: `CREATE INDEX idx_name ON table(column)` — can be done online with pt-online-schema-change for large tables
- Add query timeout at application level (JDBC: `queryTimeout`) to prevent single queries from blocking forever
- Cache frequently-read slow query results in ElastiCache/Redis

Prevention:

- Never deploy schema or query changes without running `EXPLAIN` in staging
- Set `max_execution_time` (MySQL) at DB level to auto-kill runaway queries after N seconds

- Use Read Replicas for reporting/analytics queries — never run OLAP queries on the primary
 - Set up Performance Insights with 7-day retention — invaluable for post-incident analysis
 - Review slow query log weekly, not just during incidents
-

Issue 5: RDS Read Replica Replication Lag

What happened: Read replica falling behind primary by seconds or minutes. Application reading stale data. In worst case, replica crashes or stops replicating.

How to investigate:

- CloudWatch: ReplicaLag metric on the read replica — how many seconds behind?
- High lag causes: heavy write traffic on primary outpacing replica's apply speed, large transactions, DDL on primary
- Check if replica is under-provisioned vs primary — replica needs same or better instance class
- Check binlog format: ROW format replicates more data than STATEMENT but is more consistent

How to fix:

- Scale up replica instance class to match or exceed primary — replica processes every write from primary, same CPU needed
- Throttle write rate on primary temporarily during peak lag
- For very large transactions: break into smaller batches on the primary
- If replica is too far behind and data is critical: promote replica and create a fresh one from a snapshot

Prevention:

- Always provision read replicas at the same instance class as the primary
- Monitor ReplicaLag: alarm if > 30 seconds for OLTP, > 5 minutes for batch

- Use Aurora Read Replicas instead of MySQL replicas — Aurora replicas share the same storage, so lag is typically <100ms
 - Avoid long-running DDL (ALTER TABLE) during business hours on high-write primaries
-

Issue 6: RDS Automated Backup Failing

What happened: RDS automated backup window failing silently. Discovered during DR test that latest restore point is days old.

How to investigate:

- RDS Console → Maintenance & Backups → Latest backup status
- CloudTrail: CreateDBSnapshot events — when was the last successful one?
- Check backup window: is it overlapping with peak load? RDS may skip backup if IO is saturated during the window
- Check retention period: was it accidentally set to 0? Setting retention to 0 disables automated backups

Prevention-focused fix:

- Set backup retention to minimum 7 days (35 days max) for production
- Set backup window to off-peak hours (e.g. 3-5 AM local time)
- Enable RDS Events subscription: subscribe to backup failure notifications via SNS
- Use AWS Backup service for centralized backup management with audit reports and cross-region copy
- Test restore quarterly: actually restore a snapshot to a new instance and verify data integrity

Prevention:

- CloudWatch alarm on RDS event for backup failure
- Quarterly DR drill: restore production snapshot to a new instance, run smoke tests
- Use AWS Backup with vault lock for immutable backups meeting compliance requirements
- Document RTO/RPO and verify backup strategy actually meets them

Issue 7: Deadlocks Causing Transaction Failures

What happened: Application logs showing frequent deadlock errors. Some transactions rolling back randomly. Data inconsistency concerns.

How to investigate:

- MySQL: `SHOW ENGINE INNODB STATUS\G` → LATEST DETECTED DEADLOCK section shows exact transactions and rows involved
- Enable deadlock logging: `innodb_print_all_deadlocks=1` in parameter group
- Identify which tables and rows are involved
- Common pattern: Transaction A locks row 1 then row 2, Transaction B locks row 2 then row 1 simultaneously

How to fix:

- Reorder lock acquisition to be consistent across all code paths (always lock in same table/row order)
- Use `SELECT ... FOR UPDATE` to acquire all needed locks upfront
- Reduce transaction scope — hold locks for the shortest possible time
- Application must have retry logic for deadlock errors (MySQL error 1213 is safe to retry)

Prevention:

- Add retry logic for deadlock errors at the application layer — every DB library should handle error 1213
- Use short, focused transactions — don't do HTTP calls or business logic inside a DB transaction
- Index foreign key columns — missing indexes on FK cause table-level locks during cascades
- Load test with concurrent writes to detect deadlock patterns before production

Issue 8: RDS Parameter Group Change Not Applied

What happened: Modified RDS parameter group (e.g. increased max_connections, enabled slow_query_log) but changes not taking effect.

How to investigate:

- RDS Console → Parameter Groups → check parameter Apply Status: "pending-reboot" means a static parameter was changed and requires a reboot
- Dynamic parameters apply immediately without reboot; static parameters require reboot
- Was the parameter group applied to the instance? Group must be associated with the instance, not just modified

How to fix:

- For static parameters (like max_connections): schedule a reboot during maintenance window
- For Aurora: changes apply during the next maintenance window or on manual reboot of each cluster member
- Verify the correct parameter group is associated with the DB instance in the console

Prevention:

- Know which parameters are static vs dynamic before modifying in production
- Use Aurora where possible — cluster-level dynamic parameters apply without reboot
- Schedule parameter changes with a maintenance window, not ad-hoc
- Test all parameter group changes on a staging RDS instance first

Issue 9: RDS IAM Authentication Failures

What happened: Application configured to use IAM database authentication. Suddenly getting authentication errors after working fine.

How to investigate:

- IAM auth tokens expire after 15 minutes — application must generate a fresh token per connection

- Is the application caching and reusing the token? Connection pools that cache connections reuse the token past expiration
- Is the IAM role attached to the EC2/ECS task? Check instance profile
- Does the role have `rds-db:connect` permission for the specific DB resource ARN?
- Is SSL enabled? IAM auth requires SSL connection to RDS — if SSL cert expired or disabled, auth fails

How to fix:

- Configure connection pool to regenerate IAM token before each new connection acquisition (not per query)
- Add `rds-db:connect` permission to the IAM role: `arn:aws:rds-db:region:account:dbuser/db-id/username`
- Ensure SSL is enabled and using the latest RDS CA certificate bundle
- Rotate to a fresh token immediately: tokens are generated via `generate-db-auth-token` CLI/SDK call

Prevention:

- Always treat IAM auth tokens like credentials with 15-minute TTL — generate fresh per connection, not per request
- Use RDS Proxy with IAM auth — Proxy handles token generation and connection pooling correctly
- Automate CA certificate rotation before expiry (AWS publishes rotation schedules)
- Test IAM auth token flow explicitly in staging, not just password auth

Issue 10: RDS Maintenance Window Causing Unexpected Downtime

What happened: RDS performed automatic maintenance (minor version upgrade or OS patch) during business hours. Application was down for 5-10 minutes unexpectedly.

How to investigate:

- RDS Console → Maintenance: check "Pending Maintenance" — what action was pending?

- Check configured maintenance window — was it set to a time that is now business hours in your timezone?
- Was "Auto Minor Version Upgrade" enabled? AWS may apply it during the maintenance window automatically
- Multi-AZ reduces but does not eliminate downtime for some maintenance operations

How to fix:

- Immediately reschedule maintenance window to off-peak: `aws rds modify-db-instance --preferred-maintenance-window sun:02:00-sun:04:00`
- For pending maintenance: defer it until the next window if critical operations are ongoing
- For Aurora: use zero-downtime patching (ZDP) — AWS rolls patches across replicas before primary

Prevention:

- Set maintenance window explicitly to 2-4 AM on Sunday (or lowest traffic window in your region)
- Disable Auto Minor Version Upgrade on production — test new versions in staging first, then upgrade manually
- Subscribe to RDS maintenance events via SNS — get notified before maintenance begins
- Use Aurora Multi-AZ with read replicas — Aurora ZDP patches replicas first, then promotes one, minimizing downtime
- Document your maintenance window in your team runbook and ops calendar

Lambda Production Issues

Issue 1: Lambda Timeout — Function Killed Before Completing

What happened: Function logs show `Task timed out after X.XX seconds`. Downstream operation incomplete or partially executed.

How to investigate:

- CloudWatch Logs: last line before timeout — what was the function doing?
- X-Ray trace: which downstream call (DB, HTTP, S3) consumed most of the time?
- Is the function waiting on a synchronous call to a slow downstream service?
- Is the timeout set too low for the actual work the function does?
- Check if a DB connection is being established on every invocation (cold start + connection setup overhead)

How to fix:

- Increase timeout — but diagnose root cause first; blindly increasing masks real problems
- Add per-call timeouts on HTTP clients and DB queries shorter than Lambda timeout — fail fast
- Use RDS Proxy to eliminate DB connection overhead on each invocation
- Move long-running work to async: SQS + Lambda, or Step Functions for orchestration
- Use Lambda SnapStart (Java) to reduce cold start + initialization time

Prevention:

- Set Lambda timeout = P99 observed duration × 1.5, not the maximum (15 min)
- Set HTTP client timeout = Lambda timeout - 2 seconds, so function can log before being killed
- X-Ray trace all Lambda functions in production — identify slow subsystems proactively
- For Java Spring Boot on Lambda: use Spring Cloud Function or Micronaut to reduce cold start

Issue 2: Lambda Throttling — 429 TooManyRequestsException

What happened: Lambda receiving throttling errors. Invocations failing at high volume. SQS messages backing up or API Gateway returning 429.

How to investigate:

- CloudWatch: **Throttles** metric on the Lambda function — is it > 0?

- Check account-level concurrency limit (default 1000 per region, shared across all functions)
- Check if one function has `Reserved Concurrency` set too low
- Check if Provisioned Concurrency is exhausted and burst concurrency limit was hit
- Is this an SQS-triggered Lambda? SQS will retry throttled invocations — queue depth grows

How to fix:

- Request concurrency limit increase via AWS Support (can get to 10,000+)
- Add Reserved Concurrency to critical functions — guarantees them a slice of the 1000 limit
- For SQS: throttling is safe — SQS retries automatically, just add more capacity
- For API Gateway: implement exponential backoff + retry in the client
- Add Provisioned Concurrency for latency-sensitive functions to avoid cold-start bursts

Prevention:

- Know your account-level concurrency limit and monitor ConcurrentExecutions across the account
- Reserve concurrency for each production function — don't let one runaway function starve others
- Set CloudWatch alarm: Throttles > 10 per minute → SNS alert
- Design Lambda consumers to be idempotent — safe to retry on throttle

Issue 3: Lambda Hitting Memory Limit — Silent OOM

What happened: Lambda exits with `Runtime exited with error: signal: killed` or `SIGKILL`. Function silently fails with no stack trace.

How to investigate:

- CloudWatch Logs: look for `REPORT` line — `Max Memory Used: XXXX MB`. Is it at or near the configured memory limit?
- If Max Memory Used = Memory Size configured: OOM

- For Java: heap dump on OOM is not possible in Lambda — analyze what data structures grow unboundedly
- Is the function loading large files into memory (entire S3 object, large JSON)?

How to fix:

- Increase memory allocation (also increases CPU proportionally in Lambda)
- Stream large data instead of loading into memory: use S3 GetObject as a stream, process line by line
- For Java: set Xmx = 75% of Lambda memory to leave room for JVM overhead and off-heap

Prevention:

- Always check **Max Memory Used** in REPORT logs after deployment — set memory based on actual peak usage $\times 1.5$
- Never load entire S3 objects into memory — use streaming APIs
- Set CloudWatch alarm on Lambda errors (OOM shows as an error)
- Use Lambda Power Tuning (open-source AWS tool) to find the optimal memory setting for cost and performance

Issue 4: Lambda Cold Start Latency Spike

What happened: P99 latency spikes to 5-10 seconds periodically. Users see occasional very slow requests. Corresponds to new Lambda instance initialization.

How to investigate:

- CloudWatch Logs: REPORT line shows **Init Duration: XXXX ms** — this is the cold start overhead
- X-Ray: spans show an **Initialization** segment at the start
- Cold starts happen when: first invocation, after idle (15 min+), on scale-out to new concurrency
- Java/Spring Boot have the worst cold starts (1-5 seconds) due to class loading and DI container startup

How to fix:

- Enable Provisioned Concurrency — pre-warms N instances, eliminates cold starts for those
- For Java: use Lambda SnapStart (takes a snapshot of initialized execution environment, restores on invoke)
- Move initialization code outside the handler (connection setup, config loading at module level) — runs once per environment not per invocation
- Switch to lighter frameworks: Quarkus or Micronaut (native image) has cold starts < 200ms vs Spring Boot's 3-5 seconds

Prevention:

- Use Provisioned Concurrency for latency-sensitive API-facing Lambda functions
- Keep deployment packages small — Lambda downloads and unpacks the ZIP on cold start; smaller = faster
- Avoid loading unnecessary libraries — each import adds to initialization time
- For Java: evaluate GraalVM native image compilation — eliminates JVM startup overhead entirely

Issue 5: Lambda Function Hitting /tmp Storage Limit

What happened: Function fails with "No space left on device" when writing temporary files. /tmp is Lambda's only writable filesystem.

How to investigate:

- Lambda /tmp has a default limit of 512 MB (configurable up to 10 GB now)
- Function is writing files to /tmp and either not cleaning up, or a single file exceeds the limit
- Check if multiple concurrent invocations share the same execution environment — they don't (each has its own /tmp)
- But a single warm execution environment is reused across sequential invocations — /tmp persists between invocations on the same environment

How to fix:

- Increase Lambda ephemeral storage: console → Configuration → Edit → Ephemeral storage up to 10 GB
- Clean up /tmp at the end of every invocation: `import shutil; shutil.rmtree('/tmp/workdir', ignore_errors=True)`
- For large files: stream directly to S3 without writing to /tmp — use S3 multipart upload with streaming

Prevention:

- Always clean up /tmp at the end of Lambda handler execution
- Never assume /tmp is empty at the start of invocation — execution environments are reused
- If function processes large files: stream to S3 directly, avoid /tmp entirely
- Set ephemeral storage size based on max expected file size with 50% buffer

Issue 6: Lambda Concurrency Causing DB Overload

What happened: Lambda scales to 500+ concurrent executions. Each opens a DB connection. RDS hits max connections. DB crashes.

How to investigate:

- RDS CloudWatch: DatabaseConnections spikes to max exactly when Lambda scales out
- Lambda has no built-in connection pooling — each execution environment holds one connection
- At 500 concurrent Lambdas × 1 connection each = 500 simultaneous DB connections
- RDS max_connections for db.t3.medium = ~280, for db.r5.large = ~800

How to fix:

- Deploy RDS Proxy immediately — Proxy pools Lambda connections, multiplexes many Lambda connections onto few RDS connections
- Set Lambda reserved concurrency as an immediate cap while RDS Proxy is being deployed

- RDS Proxy reduces Lambda → RDS connections from N concurrent Lambdas to a small pool (e.g. 20-50)

Prevention:

- Mandatory rule: Lambda → RDS must always go through RDS Proxy
- Size RDS Proxy max connections based on RDS instance class
- Set Lambda reserved concurrency with RDS capacity in mind even with Proxy
- Use DynamoDB instead of RDS for Lambda-heavy workloads — DynamoDB has no connection concept

Issue 7: Lambda Environment Variable Exposing Secrets

What happened: Security audit finds database passwords, API keys stored in Lambda environment variables in plaintext. Visible to anyone with `lambda:GetFunction` permission.

How to investigate:

- Lambda Console → Configuration → Environment Variables — are secrets stored here in plaintext?
- Check IAM: who has `lambda:GetFunction`? This permission exposes all environment variables
- CloudTrail: has anyone called GetFunction and retrieved these values?

How to fix:

- Move all secrets to AWS Secrets Manager or SSM Parameter Store (SecureString)
- Update Lambda to fetch secrets at runtime using SDK calls, not environment variables
- Rotate all exposed credentials immediately
- Encrypt existing environment variables with a CMK (KMS) if migration to Secrets Manager is delayed

Prevention:

- Never store secrets in Lambda environment variables in plaintext — treat them like code committed to git
 - Use Secrets Manager with automatic rotation for DB credentials
 - Cache secrets in memory within the Lambda execution environment (fetch once, reuse across invocations)
 - Restrict `lambda:GetFunction` permission to only those who genuinely need it (not all developers)
-

Issue 8: Lambda Dead Letter Queue Not Capturing Failures

What happened: Lambda async invocations failing silently. No errors in CloudWatch beyond the retry exhaustion. Lost events with no trace.

How to investigate:

- Lambda async invocations (from S3, SNS, EventBridge) retry twice by default on failure, then drop the event
- If no DLQ configured: failed events are silently discarded after retries
- CloudWatch: Lambda Errors metric shows failures, but no events captured for analysis
- Check if DLQ (SQS or SNS) was configured on the Lambda function

How to fix:

- Add SQS Dead Letter Queue to Lambda function: Configuration → Asynchronous invocation → Dead-letter queue
- Process DLQ messages: retrieve, analyze the event payload, fix the bug, replay manually
- Enable Lambda Destinations (newer, more flexible than DLQ): on-success and on-failure destinations to SQS, SNS, EventBridge, or another Lambda

Prevention:

- Every async Lambda function must have a DLQ or Destinations configured — no exceptions

- Set CloudWatch alarm: DLQ `ApproximateNumberOfMessagesVisible > 0` → immediate alert
 - Lambda Destinations > DLQ: destinations capture both success and failure, include request/response context
 - Implement idempotency in Lambda handlers to safely replay DLQ events
-

Issue 9: Lambda VPC Configuration Causing Timeout

What happened: Lambda moved into VPC to access RDS/Redis. Now timing out even on simple operations. Cold starts jumped from 500ms to 8 seconds.

How to investigate:

- Lambda in VPC used to attach an ENI per invocation — caused 10+ second cold starts (classic problem pre-2019)
- AWS fixed this with hyperplane ENIs in 2019 — but only if subnets have enough free IPs
- Check: is the subnet running out of IP addresses? Lambda in VPC needs 1 IP per concurrent execution
- Check: does the Lambda have a route to the internet? Lambda in private subnet loses internet access unless there's a NAT Gateway
- Check Security Group: does it allow outbound to RDS port (3306) and return traffic?

How to fix:

- Use larger subnets (/24 or larger) for Lambda in VPC — ensure enough IPs for peak concurrency
- Add NAT Gateway if Lambda needs internet access (e.g. to call external APIs)
- Add VPC Endpoints for AWS services (S3, DynamoDB, Secrets Manager) — avoid internet for AWS service calls
- Verify Security Group allows outbound on 3306/6379 to RDS/Redis SG

Prevention:

- Always use /24 or larger subnets for Lambda VPC deployment

- Add VPC endpoints for every AWS service the Lambda calls — eliminates internet dependency and reduces latency
 - Prefer placing Lambda outside VPC when possible — use RDS Proxy (public endpoint with TLS) to access RDS
 - Monitor ENI usage: `aws ec2 describe-network-interfaces --filters "Name=interface-type,Values=lambda"`
-

Issue 10: Lambda Handling Duplicate Events (Non-Idempotency)

What happened: Lambda processes payments or sends emails. Occasionally, the same event is processed twice. Customers double-charged or double-emailed.

How to investigate:

- Lambda guarantees at-least-once delivery for async invocations — duplicate invocations are normal
- SQS Standard: at-least-once delivery — Lambda may receive the same message twice
- When does duplication happen: Lambda retries on partial batch failure, SQS visibility timeout < Lambda execution time, Lambda itself retries after timeout
- Check application logs for same event ID processed multiple times with timestamps close together

How to fix:

- Implement idempotency: use the event's unique ID (SQS MessageId, EventBridge event ID) as an idempotency key
- Store processed IDs in DynamoDB or Redis with a TTL: check before processing, write after
- Use AWS Lambda Powertools Idempotency module (Python/Java) — handles this pattern with DynamoDB automatically
- For payments: use payment provider's idempotency keys (Stripe, Razorpay all support this)

Prevention:

- Design all Lambda handlers to be idempotent from day one — assume every event may be delivered more than once
 - Use SQS FIFO with message deduplication ID for exactly-once processing when ordering matters
 - Idempotency key TTL should match the maximum retry window (24 hours is safe for most cases)
 - Test duplicate invocation in staging by manually invoking with the same payload twice
-

Auto Scaling Group Production Issues

Issue 1: ASG Not Scaling Out During Traffic Spike

What happened: Traffic surged, CPU hit 100%, but ASG did not add instances. Users experienced timeouts.

How to investigate:

- ASG Activity History — any scale-out events? If none, the alarm never fired
- CloudWatch: check the scaling metric (CPUUtilization) — did it breach the threshold?
- Is the alarm in ALARM state? If it went from OK → INSUFFICIENT_DATA → ALARM it may have been suppressed
- Is the ASG at MaxCapacity? If desired = max, no scaling is possible
- Was there a cooldown period active from a previous scale event blocking the new one?
- Is the scaling policy Target Tracking, Step, or Simple? Simple and Step have longer reaction times

How to fix:

- Increase MaxCapacity immediately in the console
- Manually increase desired capacity as an emergency measure
- Switch from Simple/Step to Target Tracking — reacts continuously without alarm delays

- Reduce cooldown period (default 300s is often too long for modern containerized apps)

Prevention:

- Set MaxCapacity generously — cost of over-provisioning briefly is far less than downtime
- Use Target Tracking with CPUUtilization target of 60% — leaves headroom for traffic bursts
- Load test the ASG scaling behavior in staging: confirm it scales within 2 minutes of sustained load
- Set CloudWatch alarm: desired capacity = max capacity → alert the team before it's a problem

Issue 2: ASG Scaling In Aggressively — Terminating Busy Instances

What happened: ASG scale-in terminated an instance that was serving active connections. Users mid-session got dropped. Database transactions orphaned.

How to investigate:

- ASG Activity History: scale-in event shows instance terminated
- Was Connection Draining (deregistration delay) configured on the ALB target group?
- Default deregistration delay is 300 seconds — did the scale-in happen faster?
- Did the instance finish in-flight requests before termination?
- Check scale-in protection: was it enabled on any instances?

How to fix:

- Enable Instance Scale-In Protection on instances processing long transactions (via lifecycle hooks)
- Configure ALB target group deregistration delay appropriately (30s for fast APIs, 300s for long operations)
- Add ASG Lifecycle Hook on termination: sends instance to Terminating:Wait state, allowing graceful shutdown

- Application should handle SIGTERM gracefully: finish current requests, close DB connections, flush logs

Prevention:

- Always configure Termination Lifecycle Hook for production ASGs
- Set deregistration delay in ALB target group to max expected request duration + 30s
- Implement graceful shutdown in your application: catch SIGTERM, stop accepting new requests, drain current ones
- For stateful operations: use scale-in protection programmatically for instances processing jobs

Issue 3: Launch Template / Configuration Errors — Instances Fail to Launch

What happened: ASG trying to scale out but instances immediately fail to launch or fail health checks. Rapid launch-fail-terminate cycle.

How to investigate:

- ASG Activity History: "Failed to launch instance. Reason: ..."
- Common reasons: AMI not found (deregistered), instance type not available in AZ, spot price too low, IAM instance profile not found, security group deleted
- If instances launch but fail health checks: check UserData script errors in `/var/log/cloud-init-output.log`
- If AMI issue: `aws ec2 describe-images --image-ids <ami-id>` — does it exist in this region?

How to fix:

- Update launch template with a valid AMI, correct instance type, existing security groups
- Test UserData script on a standalone EC2 first before putting it in a launch template
- For spot: add fallback on-demand instances in mixed instances policy
- For AZ availability issues: add more AZs or instance types to the launch template

Prevention:

- Use launch template versioning — pin ASG to a specific tested version, not \$Latest
 - Never delete AMIs that are referenced by a launch template without updating the template first
 - Test launch templates by manually launching an instance from the template before deploying to ASG
 - Use mixed instances policy with 2-3 instance type families — if one type is unavailable, ASG uses another
-

Issue 4: ASG Instances Not Registering With ALB

What happened: New ASG instances launch successfully but never appear as healthy in ALB target group. Traffic not routed to them.

How to investigate:

- ALB Target Group → Targets tab: are new instances showing? Status: unused, initial, healthy, or unhealthy?
- "unused": instance not yet registered. "initial": health check in progress. "unhealthy": health check failing
- Is the ASG linked to the correct target group? Check ASG → Load Balancing configuration
- Is the health check path correct? Does the app respond with 200 on that path?
- Is the health check port correct? App on 8080, health check on 80?
- Security group of instance: does it allow inbound from ALB security group on the app port?

How to fix:

- Add inbound rule to instance security group: source = ALB SG, port = app port (8080/443)
- Correct health check path and port in target group
- Manually register the instance in the target group to test health check independently
- Check app startup: `curl http://localhost:8080/actuator/health` — does it return 200?

Prevention:

- Always test health check endpoint locally before deploying: `curl -v http://instance-ip:port/health`
- Use security group referencing (ALB SG → Instance SG) not CIDR ranges — works dynamically as IPs change
- Set health check grace period in ASG = app startup time + 30 seconds to avoid premature unhealthy marking
- Automate smoke test on new AMI: launch, wait, check health endpoint, validate response format

Issue 5: Scaling Policy Thrashing — Constant Scale Out and In

What happened: ASG scales out, then immediately scales back in, then out again repeatedly. Instances constantly launching and terminating. Wasted cost and instability.

How to investigate:

- ASG Activity History: alternating ScaleOut and ScaleIn events every few minutes
- CloudWatch metric: CPUUtilization oscillating around the threshold (e.g. 70% threshold, metric going 65-75-65-75)
- Cooldown period may be too short to prevent oscillation
- Target Tracking without scale-in cooldown causes aggressive scale-in after brief traffic drop

How to fix:

- Increase scale-in cooldown period (default 300s — try 600s or more)
- For Target Tracking: disable scale-in temporarily (`disableScaleIn: true`) and handle scale-in manually
- Use a lower target for scale-out (e.g. CPU 50%) so instances have headroom before scale-in triggers
- Implement predictive scaling to smooth out oscillation with forecast-based capacity

Prevention:

- Set different thresholds for scale-out and scale-in: scale out at 70% CPU, scale in at 40% CPU — creates a buffer zone
 - Use Step Scaling with distinct step ranges rather than a single threshold
 - Use longer evaluation periods on CloudWatch alarms (3 of 5 datapoints) to smooth noise
 - Predictive Scaling + Dynamic Scaling together: predictive handles known patterns, dynamic handles surprises
-

Issue 6: ASG Ignoring AZ Balance — All Instances in One AZ

What happened: ASG gradually drifted so that 8 of 10 instances are in us-east-1a, only 2 in us-east-1b. When us-east-1a had a brief AZ degradation, 80% of capacity was lost.

How to investigate:

- EC2 Console → ASG → Instance Management: check AZ distribution
- This happens when one AZ has more capacity/cheaper spot pricing, and ASG preferentially launches there
- Or: instances in one AZ were manually terminated and ASG didn't rebalance
- Check if ASG has AZRebalance activity in Activity History

How to fix:

- Trigger manual AZ rebalance: ASG → Actions → Rebalance Availability Zones
- ASG rebalancing launches in underrepresented AZs first, then terminates from over-represented ones (with connection draining)

Prevention:

- Enable AZ Rebalance in ASG — it's on by default but verify it hasn't been disabled
 - Specify all desired AZs explicitly in ASG configuration
 - CloudWatch alarm: if instance count in any AZ falls to 0 → alert
 - For critical workloads: set minimum healthy percentage per AZ using capacity distribution strategy in launch template
-

Issue 7: Warm Pools Not Working as Expected

What happened: Configured Warm Pool to speed up scaling. Instances still take 3-4 minutes to become healthy after scale-out event.

How to investigate:

- Warm Pool keeps pre-initialized instances in Stopped or Running state
- Check Warm Pool size: is it large enough to cover typical scale-out events?
- Are warm pool instances actually stopping and waiting? Check their state in EC2 console
- Is the lifecycle hook on the warm pool (WarmPoolMinSize, WarmPoolMaxGroupPreparedCapacity) configured correctly?
- If instances enter warm pool but still run UserData on transition to active: lifecycle hook must handle this

How to fix:

- Set WarmPoolMinSize to cover your largest expected scale-out increment
- Use Hibernate state (not Stopped) if supported — faster resume as OS state is preserved
- Ensure application configuration is baked into AMI, not applied via UserData on active transition
- Set lifecycle hook on instance-entering-warm-pool to pre-initialize what's needed

Prevention:

- Warm pools are most effective when combined with pre-baked AMIs — use them together
- Size the warm pool based on your largest historical scale-out event plus 20% buffer
- Monitor WarmPool instance count: if it falls to 0 frequently, the pool is undersized
- For Java apps: combine warm pool with application pre-warming (hit health endpoint to trigger JIT compilation)

Issue 8: ASG Termination Policy Terminating Wrong Instances

What happened: ASG scale-in terminated a critical instance running a long batch job, causing data loss. Expected it to terminate the newest instance first.

How to investigate:

- Default ASG termination policy: OldestLaunchTemplate → OldestInstance → closest to billing hour
- This may not align with your desired behavior
- Check if the terminated instance had scale-in protection enabled — it should have for long-running jobs
- Check active termination policy in ASG settings

How to fix:

- Enable instance-level scale-in protection programmatically when a job starts: `aws autoscaling set-instance-protection`
- Remove protection when job completes
- Set ASG termination policy to `NewestInstance` to always terminate freshest instances first (safer for stateful apps)
- Use lifecycle hooks to gracefully drain before termination

Prevention:

- Use `set-instance-protection` API in your job scheduler: protect on start, unprotect on completion
- For batch workloads in ASG: use separate ASG from web-serving ASG so scale-in policies don't conflict
- Always add a Terminating lifecycle hook for graceful shutdown
- Document and review the termination policy — teams often don't know what policy is active

Issue 9: Predictive Scaling Not Kicking In Early Enough

What happened: Predictive scaling enabled but traffic spikes still caught ASG underprepared. Scaling happened reactively after load already hit.

How to investigate:

- CloudWatch: predictive scaling forecast — was the spike in the pattern it learned?
- Predictive scaling learns from 14 days of historical CloudWatch data — new ASGs won't have enough data
- Is the load pattern irregular enough that ML forecast missed it?
- Was "Forecast Only" mode enabled instead of "Forecast and Scale"?

How to fix:

- Switch from "Forecast Only" to "Forecast and Scale" mode
- Combine Predictive Scaling with Dynamic Scaling (Target Tracking) — predictive handles pattern-based load, dynamic handles unexpected spikes
- Pre-schedule a Scheduled Scaling action for known peak times (product launches, marketing campaigns) that predictive won't know about

Prevention:

- Always combine Predictive + Dynamic scaling — never rely on either alone
- For known one-time events (launches, sales): add a Scheduled Scaling action explicitly
- Review predictive forecast graph weekly — if actual traffic consistently exceeds forecast, increase buffer
- Give ASG 14+ days of history before relying on predictive scaling — use Scheduled in the interim

Issue 10: ASG Scaling Activity Stuck in Pending

What happened: Scale-out event triggered but instances stuck in "Pending" state for 10+ minutes. ASG Activity History shows no completion.

How to investigate:

- Check Lifecycle Hooks: is there a custom lifecycle hook in **Pending:Wait** state that is not completing?
- Lifecycle hooks wait for **CompleteLifecycleAction** call — if the Lambda/script handling the hook crashed, instance stays in Pending indefinitely

- Check if the hook heartbeat timeout (default 1 hour) has been exceeded
- Check EC2 instance state in console — is it running, or stuck in pending?
- Check VPC capacity: is there enough IP address space in the subnet for new ENIs?

How to fix:

- Manually complete stuck lifecycle hook: `aws autoscaling complete-lifecycle-action --lifecycle-action-result CONTINUE`
- Fix the Lambda/script handling the lifecycle hook and redeploy
- Abandon stuck instance if needed: `aws autoscaling complete-lifecycle-action --lifecycle-action-result ABANDON`
- Check subnet free IP addresses: `aws ec2 describe-subnets` → AvailableIpAddressCount

Prevention:

- Always add error handling and a DLQ to any Lambda handling lifecycle hooks
- Set a reasonable lifecycle hook heartbeat timeout (15-30 min) not the default 1 hour
- Lifecycle hook handlers should call CompleteLifecycleAction in both success AND failure paths — never let it time out
- CloudWatch alarm: if `GroupPendingInstances > 0` for > 10 minutes → alert

ECS Production Issues

Issue 1: ECS Task Keeps Restarting in a Loop

What happened: ECS service shows tasks launching and stopping repeatedly. Service never reaches stable state. `RUNNING (1/2)` never becomes `(2/2)`.

How to investigate:

- ECS Console → Service → Events tab: "service has been unable to place a task" or "task stopped with exit code X"
- CloudWatch Logs for the container: what error appears before it stops?

- Exit code 1: application crash. Exit code 137: OOM killed. Exit code 143: SIGTERM (graceful shutdown triggered)
- Check task definition: is the Docker image tag valid? Does it exist in ECR?
- Check health check: `startPeriod` too short for Spring Boot startup? Health check fails before app is ready → task killed

How to fix:

- Fix application crash (check stack trace in CloudWatch Logs)
- Increase memory/CPU in task definition for OOM (137)
- For Spring Boot: set `startPeriod` = 120 seconds in task definition health check
- For image pull failure: check ECR repository permissions on task execution role

Prevention:

- Set `startPeriod` in health check to cover worst-case startup time
- Always tag images with version hash, not just `latest` — prevents ambiguity
- Set CloudWatch alarm on `service_running_task_count < desired_count`
- Test Docker image locally with same memory limits before deploying to ECS

Issue 2: ECS Cannot Place Tasks — Insufficient Resources

What happened: Scale-out triggered but ECS cannot place new tasks. Activity log shows "Unable to place task. No container instances met all of its requirements."

How to investigate:

- This applies to ECS on EC2 (not Fargate — Fargate has no placement constraint)
- CloudWatch: ClusterReservation for CPU and memory — is it near 100%?
- Check task CPU/memory vs available EC2 instance capacity
- Check placement constraints: if task requires specific instance attributes (AZ, instance type) not available
- Fragmentation: enough total memory exists but not in a single instance that fits the task

How to fix:

- Add more EC2 instances to the ECS cluster (if using EC2 launch type) or switch to Fargate
- Use ECS Capacity Providers with Auto Scaling — cluster scales EC2 automatically when tasks cannot be placed
- Review placement constraints — are they more restrictive than necessary?
- Switch to Fargate for variable workloads — eliminates capacity management entirely

Prevention:

- Use Fargate for production ECS workloads unless you have specific EC2 requirements
- If using EC2: configure ECS Capacity Provider with Managed Scaling — ECS drives ASG scaling automatically
- Monitor ClusterReservation: alarm if CPU or memory > 75%
- Use binpack placement strategy to maximize density and minimize fragmentation

Issue 3: ECS Fargate Task Failing to Pull Image

What happened: New ECS deployment failing. Tasks stop with "CannotPullContainerError". Rollback triggered automatically.

How to investigate:

- ECS Service events: "CannotPullContainerError: Error response from daemon: pull access denied"
- Check: does the ECS task execution role have `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`?
- Check: is the ECR repository in the same region as the ECS cluster? Cross-region requires the registry URL to include the region
- Check: is the image tag correct? Does it exist in ECR? `aws ecr list-images --repository-name <repo>`
- For Fargate in private subnet: does it have a VPC Endpoint for ECR (com.amazonaws.region.ecr.dkr) or NAT Gateway for internet access?

How to fix:

- Add ECR permissions to task execution role
- Add VPC Endpoint for ECR if tasks are in a private subnet with no NAT
- Verify and correct image URI and tag in task definition
- For private subnet without VPC Endpoint: add NAT Gateway route

Prevention:

- Add ECR VPC Endpoints (ecr.api + ecr.dkr + S3 gateway endpoint) in all private subnets used by ECS
- Use ECR image scanning and lifecycle policies — old/vulnerable images get cleaned, reducing confusion over valid tags
- Pin task definitions to specific image digest (SHA256) not just tag — prevents "tag moved" surprises
- Test image pull from within VPC before deploying: `docker pull <ecr-uri>` from an EC2 in same VPC/subnet

Issue 4: ECS Blue/Green Deployment Stuck — Not Completing

What happened: CodeDeploy blue/green ECS deployment started, replacement tasks healthy, but deployment stuck in "Waiting for termination of original tasks" for hours.

How to investigate:

- CodeDeploy deployment detail page: which step is stuck?
- If stuck at "Replace original tasks": the original task set may have active connections not draining
- Check ALB deregistration delay: if set to 3600 seconds, CodeDeploy waits that long
- Is there a manual approval step in the deployment configuration that someone forgot to click?
- Are there long-lived WebSocket or SSE connections keeping the original tasks alive?

How to fix:

- Reduce ALB target group deregistration delay to a reasonable value (30-60 seconds for REST APIs)
- Check for and terminate long-lived connections (WebSocket clients should implement reconnect logic)
- If stuck on approval: approve via CodeDeploy console or CLI
- Force terminate original task set via CodeDeploy if safe: stop deployment and roll forward

Prevention:

- Set deregistration delay appropriate to your connection type: 30s for REST, longer only for long-lived connections
- Implement connection draining in application: on SIGTERM, stop accepting new connections, finish existing ones within 30s
- Configure reasonable CodeDeploy deployment timeouts — don't allow stuck deployments to block pipeline for hours
- Use ECS rolling update (not blue/green) for routine deployments; blue/green for high-risk changes

Issue 5: ECS Service Discovery Not Resolving

What happened: Service A cannot reach Service B using Cloud Map service discovery DNS name. Connection refused or DNS resolution failure.

How to investigate:

- Test DNS resolution from within the container: `nslookup service-b.local` or `dig service-b.namespace.local`
- Check Cloud Map: is Service B registered? Are its health checks passing?
- ECS registers tasks in Cloud Map automatically — are tasks healthy in ECS? Unhealthy tasks are deregistered
- Check VPC DHCP options: is the VPC using Amazon's DNS (Route53 Resolver at 169.254.169.253)?
- Check Security Group: does Service A have outbound and does Service B allow inbound from Service A's SG?

How to fix:

- Fix Service B's health to get tasks registered in Cloud Map
- Add Security Group rule: Service B inbound from Service A SG on the service port
- Verify namespace and service name in the discovery URL (case-sensitive)
- For immediate fix: use ALB DNS name as a fallback while Cloud Map is debugged

Prevention:

- Always use ALB in front of ECS services for service-to-service communication in production — ALB is more observable and robust than direct service discovery
- If using Cloud Map: set health check grace period to cover startup time
- Test service discovery in staging explicitly: deploy both services, curl from container A to B's discovery name
- Monitor Cloud Map health check failures via CloudWatch

Issue 6: ECS Fargate Running Out of IPs in Subnet

What happened: New Fargate tasks failing to launch. ECS events show "Unavailable: no available IPs". Cluster appears to have capacity but tasks cannot start.

How to investigate:

- Each Fargate task gets its own ENI (Elastic Network Interface) = 1 IP address consumed per task
- `aws ec2 describe-subnets --subnet-ids <id>` → AvailableIpAddressCount = 0 or very low
- Check how many tasks are running vs subnet size: /24 = 251 IPs, but some reserved by AWS

How to fix:

- Add additional subnets from a larger CIDR range to the ECS service VPC configuration
- Scale down non-critical tasks temporarily to free IPs
- Request a CIDR expansion from the VPC team and add new subnets

Prevention:

- For Fargate: use /23 or /22 subnets (512-1023 IPs) not /24 when running many tasks
- Monitor subnet AvailableIpAddressCount: alarm when < 20% of total
- Spread tasks across multiple subnets in multiple AZs — both for HA and IP space
- Use multiple subnets in ECS service networking config — ECS distributes across all listed subnets

Issue 7: ECS Container Logs Not Appearing in CloudWatch

What happened: ECS tasks running but CloudWatch log group has no entries. Cannot diagnose issues.

How to investigate:

- Check task definition log configuration: is `awslogs` driver configured? Is the log group name and region correct?
- Check task execution role: does it have `logs:CreateLogGroup`, `logs:CreateLogStream`, `logs:PutLogEvents`?
- Is the log group created? CloudWatch Logs → verify log group exists (ECS can auto-create it if `awslogs-create-group=true`)
- For Fargate: logs go directly to CloudWatch, no ECS agent involved — execution role permissions are critical
- For EC2 launch type: is the ECS agent running on the host? `systemctl status ecs`

How to fix:

- Add CloudWatch Logs permissions to task execution role
- Add `awslogs-create-group: "true"` to log configuration in task definition
- Verify region matches in log configuration — logs going to us-east-1 when cluster is ap-south-1 is a common mistake

Prevention:

- Use the AWS-managed execution role policy `AmazonECSTaskExecutionRolePolicy` as a base — it includes CloudWatch Logs permissions
- Add `awslogs-create-group: "true"` to all task definitions — auto-creates log group if missing
- Verify logging in staging before production: deploy, exec into container, run `echo test` and check CloudWatch
- Use Container Insights for ECS — richer metrics and centralized logging configuration

Issue 8: ECS Tasks Not Getting Updated Secrets From Secrets Manager

What happened: Rotated database password in Secrets Manager. ECS tasks are still using old password and failing to connect to RDS.

How to investigate:

- ECS injects Secrets Manager values as environment variables at task start time — they are not live-refreshed
- Running tasks still have the old password in their environment — they were started before rotation
- New tasks launched after rotation will get the new value
- If connection pool is caching the connection: even restarting tasks may not help until existing connections are evicted

How to fix:

- Force new task deployment: `aws ecs update-service --cluster <c> --service <s> --force-new-deployment`
- This cycles all tasks, new tasks pick up the new secret value
- For zero-downtime: configure RDS Proxy — Proxy handles credential rotation transparently without app restarts

Prevention:

- Use RDS Proxy with Secrets Manager integration — Proxy rotates credentials automatically with zero app impact

- For non-DB secrets: implement secrets fetching at runtime (within the handler or at startup) not just at task launch
 - Coordinate secret rotation with deployment: rotate secret → force new deployment
 - Use Secrets Manager rotation Lambda that also triggers a new ECS deployment
-

Issue 9: ECS Service Autoscaling Not Working

What happened: ECS service autoscaling configured on CPU metric but service not scaling despite high CPU utilization.

How to investigate:

- CloudWatch: is the ECS CPU metric (CPUUtilization at service level) breaching the alarm threshold?
- Is Application Auto Scaling registered for the ECS service? Check Application Auto Scaling scalable targets
- Is the CloudWatch alarm in ALARM state? Check alarm history
- Is the service at MaxCapacity? If current running = max, scale-out won't happen
- Is the ECS service using Fargate spot? Spot capacity may not be available

How to fix:

- Register scalable target: `aws application-autoscaling register-scalable-target --service-namespace ecs --resource-id service/<cluster>/<service>`
- Create scaling policy linked to the correct CloudWatch metric
- Increase MaxCapacity in Application Auto Scaling
- Verify metric namespace: ECS service metrics are under `AWS/ECS` with dimensions ClusterName + ServiceName

Prevention:

- Set up ECS autoscaling via CloudFormation/Terraform — manual console setup is error-prone
- Set MaxCapacity to 3-5× the normal running count — don't constrain it to current needs

- Test autoscaling explicitly: run a load test and confirm scale-out happens within 3 minutes
 - CloudWatch alarm: ECS `RunningTaskCount < DesiredTaskCount` for > 5 minutes → alert
-

Issue 10: ECS Exec (Container Shell) Not Working for Debugging

What happened: Team trying to `aws ecs execute-command` into a running container for debugging. Getting "Execute command failed" error.

How to investigate:

- ECS Exec requires SSM agent running inside the container — is it included in the Docker image?
- Task definition must have `enableExecuteCommand: true`
- Task role must have SSM permissions: `ssmmessages:CreateControlChannel`, `ssmmessages:CreateDataChannel`, `ssmmessages:OpenControlChannel`, `ssmmessages:OpenDataChannel`
- For Fargate: ECS agent version must support ECS Exec (platform version 1.4.0+)
- Is the task running? ECS Exec only works on running tasks, not stopped ones

How to fix:

- Add SSM agent to Docker image: for Amazon Linux base images it's pre-installed; for slim images add `amazon-ssm-agent` package
- Add SSM permissions to task role
- Set `enableExecuteCommand: true` in service definition and force new deployment
- For Fargate: set platform version to `LATEST` (1.4.0+)

Prevention:

- Standardize on a base image that includes SSM agent for all ECS services
- Enable ECS Exec on all services in dev/staging — production use is audited via CloudTrail
- CloudTrail logs every ECS Exec session — useful for security audit

- Document ECS Exec as the standard debugging procedure to replace unsafe approaches like opening debug ports

API Gateway Production Issues

Issue 1: API Gateway 502 Bad Gateway

What happened: Clients receiving 502 Bad Gateway responses. Application returning errors intermittently or constantly.

How to investigate:

- CloudWatch: `5XXError` metric and `IntegrationLatency` — is the integration timing out or throwing an exception?
- 502 from API Gateway specifically means the integration (Lambda, HTTP backend) returned an invalid response
- For Lambda integration: Lambda threw an unhandled exception, OR response format is wrong (missing statusCode field)
- For HTTP integration: backend server returned an invalid HTTP response or closed connection unexpectedly
- Check Lambda logs in CloudWatch for unhandled exceptions or malformed response objects

How to fix:

- For Lambda: ensure every response is `{ "statusCode": 200, "headers": {}, "body": "..." }` — missing statusCode causes 502
- Add global try/catch in Lambda handler: return `{ statusCode: 500, body: JSON.stringify({ error: "Internal error" }) }` instead of throwing
- For HTTP backend: investigate backend server health
- Enable API Gateway access logs to see exact request/response for debugging

Prevention:

- Implement a Lambda middleware/wrapper that catches all errors and returns a properly formatted response
 - Enable API Gateway execution logs (ERROR level minimum) — logs exact reason for 502
 - Write integration tests that validate response format from Lambda before deploying
 - Set up CloudWatch alarm: 5XXError rate > 1% → SNS alert
-

Issue 2: API Gateway 504 Gateway Timeout

What happened: Clients receiving 504 Gateway Timeout. API Gateway has a hard 29-second integration timeout that cannot be increased.

How to investigate:

- CloudWatch: **IntegrationLatency** near or at 29000ms confirms timeout
- Which integration is slow: Lambda function, HTTP backend, VPC Link?
- For Lambda: X-Ray trace shows where time is spent — DB query? External HTTP call? S3?
- Is Lambda hitting a cold start + slow DB connection setup?

How to fix:

- Optimize the integration to respond within 29 seconds (ideally < 10 seconds)
- Implement async pattern: client calls API → API writes to SQS → returns 202 Accepted → Lambda processes → client polls another endpoint for result
- Use WebSocket API for long-running operations with progress updates
- For Lambda: use RDS Proxy to eliminate connection setup time

Prevention:

- Design APIs to respond within 3-5 seconds maximum — 29 seconds is too close to timeout for reliability
- Use async patterns (SQS + polling, WebSocket, or Server-Sent Events) for anything that takes > 5 seconds

- X-Ray trace all API Gateway → Lambda integrations to find slow paths before they hit timeouts
 - Set Lambda's own timeout to 25 seconds max for API Gateway-triggered functions — allows Lambda to return a proper error before API Gateway cuts the connection
-

Issue 3: API Gateway 429 Throttling — Legitimate Traffic Blocked

What happened: Clients receiving 429 Too Many Requests. Legitimate user traffic being throttled. Not a DDoS.

How to investigate:

- CloudWatch: **Count** (total requests) and check against configured throttle limits
- Default account limit: 10,000 RPS with 5,000 burst (per region, across all APIs)
- Stage-level throttle: was a low default throttle set on the API stage?
- Method-level throttle: is a specific endpoint throttling while others are fine?
- Usage Plans: is a specific API key hitting its configured quota/rate limit?

How to fix:

- Request account-level throttle increase via AWS Support
- Increase stage-level or method-level throttle limits in API Gateway settings
- Review Usage Plans: increase rate/quota for affected API keys
- For burst traffic: implement client-side retry with exponential backoff + jitter
- Implement caching on GET endpoints to reduce load on the integration

Prevention:

- Set method-level throttle limits deliberately based on what the backend can handle — don't leave at API-level defaults
- Use API Gateway caching (TTL 300s on GET) for frequently accessed read-only endpoints
- Use Usage Plans to tier API consumers — protect backend from one client over-consuming

- Load test your API and integration together to find the actual throughput ceiling before going live

Issue 4: CORS Errors on API Gateway

What happened: Browser JavaScript application getting CORS errors on API Gateway endpoints. Preflight OPTIONS requests failing.

How to investigate:

- Browser DevTools: OPTIONS preflight returns 403 or missing `Access-Control-Allow-Origin` header
- Is CORS enabled on the API Gateway resource? Console → Resource → Enable CORS
- For Lambda proxy integration: Lambda must return CORS headers in its response, not just API Gateway
- Is the specific HTTP method (PUT, DELETE) included in Access-Control-Allow-Methods?
- After enabling CORS, was the API redeployed? Changes require deployment to take effect

How to fix:

- For REST API: enable CORS on each resource, then redeploy the stage
- For Lambda proxy: add to Lambda response headers: `"Access-Control-Allow-Origin": "https://app.example.com"`, `"Access-Control-Allow-Headers": "Content-Type, Authorization"`, `"Access-Control-Allow-Methods": "GET, POST, PUT, DELETE, OPTIONS"`
- Handle OPTIONS method explicitly in Lambda if using proxy integration
- For HTTP API (simpler): CORS configuration is centralized and not per-resource

Prevention:

- Use HTTP API instead of REST API for new projects — CORS configuration is centralized and simpler
- Always redeploy after CORS changes — API Gateway changes don't auto-deploy

- Test CORS in staging with actual browser request, not just curl (curl doesn't send preflight)
- Never use `"*"` for Allow-Origin with authenticated APIs — always specify exact origin(s)

Issue 5: API Gateway Authorizer Rejecting Valid Tokens

What happened: Cognito or Lambda authorizer returning 401 Unauthorized for tokens that appear valid. Users randomly getting logged out.

How to investigate:

- CloudWatch Logs for the Lambda authorizer: is it throwing an exception? Is the token parsing failing?
- For Cognito authorizer: is the JWT token expired? Default Cognito access token TTL = 1 hour
- Is the authorizer cache enabled? Cached authorization decisions may be stale (default TTL 300s)
- Is the token clock skew an issue? JWT `iat` / `exp` compared against server time — if server clock is off, valid tokens rejected
- For Lambda authorizer: check the IAM policy format returned — malformed policy causes 500, not 401

How to fix:

- For expired tokens: implement client-side token refresh logic using refresh token
- Disable or reduce authorizer cache TTL if caching stale decisions
- For Lambda authorizer: fix the policy format: `{ "principalId": "user", "policyDocument": { "Version": "2012-10-17", "Statement": [...] } }`
- Fix server NTP if clock skew is causing valid token rejection

Prevention:

- Set authorizer cache TTL to match your security requirements — 0 for sensitive APIs, 300s for performance

- Implement token refresh in all clients before token expiry — refresh at 80% of TTL
 - Validate Lambda authorizer policy format in staging using API Gateway test invoke
 - Set CloudWatch alarm: AuthorizerError metric > 0 → investigate
-

Issue 6: API Gateway Stage Variables Not Updating

What happened: Updated a stage variable (e.g. Lambda alias, backend URL) in API Gateway. API still routing to old integration.

How to investigate:

- Stage variables are environment-specific config for API Gateway stages
- Changes to stage variables require a new deployment of the stage to take effect
- Check: was the stage redeployed after the variable change?
- For Lambda integrations using alias: is the Lambda function ARN in the integration using `${stageVariables.lambdaAlias}`?

How to fix:

- Redeploy the stage: API Gateway → Stage → Deploy
- Verify stage variable value after deployment
- For Lambda alias routing: test with API Gateway test invoke using specific stage

Prevention:

- Use Infrastructure as Code (SAM, CDK, Terraform) for API Gateway — variable changes are deployed automatically as part of the pipeline
 - Never make API Gateway changes in the console without also deploying — console changes are pending until deployed
 - Use canary deployments in API Gateway stages for gradual rollouts with stage variables controlling traffic split
-

Issue 7: API Gateway Caching Returning Wrong Data

What happened: GET endpoint returning stale or incorrect data to some users. Cache returning another user's response.

How to investigate:

- Is API Gateway caching enabled on this stage/method?
- What is the cache key? Default cache key is the URL path + query string — does NOT include Authorization header or user ID
- If user-specific data is cached without user identity in the cache key, User A may get User B's data
- Check cache TTL — is it too long for this data type?

How to fix:

- Add Authorization header or user ID query parameter to the cache key: API Gateway → Method → Cache Key Parameters
- Reduce cache TTL for user-specific or frequently-changing data
- Disable caching entirely for endpoints returning user-specific data unless cache key is properly configured
- Invalidate the cache immediately: `aws apigateway flush-stage-cache --rest-api-id <id> --stage-name <stage>`

Prevention:

- Enable caching only for public, non-user-specific GET endpoints
- For user-specific endpoints: either disable caching or always include user identity in cache key
- Document which endpoints have caching enabled and with what TTL — teams forget this configuration exists
- Test caching behavior explicitly: call as User A, call as User B, verify responses are different

Issue 8: API Gateway WebSocket Connections Dropping

What happened: WebSocket clients getting disconnected randomly. Long-lived connections not maintaining. Real-time features breaking.

How to investigate:

- API Gateway WebSocket has a 10-minute idle connection timeout (no messages in either direction)
- API Gateway also has a 2-hour maximum connection duration regardless of activity
- Check if client is sending keep-alive (ping) messages every < 10 minutes
- CloudWatch: ConnectionCount metric dropping unexpectedly
- Check Lambda disconnect handler — is it receiving and logging disconnection reasons?

How to fix:

- Implement client-side ping: send a ping frame or application-level heartbeat every 5 minutes
- Implement client-side auto-reconnect with exponential backoff + session state restore
- For 2-hour limit: implement graceful reconnection before the 2-hour mark in the client

Prevention:

- Treat WebSocket connections as ephemeral — design for reconnection from day one
- Store connection state in DynamoDB (connectionId → userId + state) so reconnects restore context
- Implement both client-side heartbeat AND reconnect logic — defend against both idle timeout and 2-hour limit
- CloudWatch alarm: ConnectionCount drops by > 50% in 1 minute → investigate

Issue 9: API Gateway VPC Link Connectivity Failure

What happened: API Gateway with VPC Link to internal NLB stopped working. 503 Service Unavailable responses.

How to investigate:

- VPC Link uses NLB to route from API Gateway into your VPC
- Check VPC Link status: API Gateway → VPC Links — is status "AVAILABLE" or "FAILED"?
- Check NLB health: are targets registered and healthy?

- Check Security Groups on the NLB targets — NLB does not have SGs itself, but target instances do
- NLB preserves client source IP — target security groups must allow traffic from NLB's subnet CIDR, not from NLB SG

How to fix:

- Fix target registration and health checks on the NLB
- Add inbound rule to target SG allowing traffic from NLB subnet CIDRs (not SG, since NLB doesn't have one)
- If VPC Link itself is FAILED: delete and recreate (VPC Link creation takes 2-4 minutes)
- Verify the NLB ARN referenced in VPC Link is correct and the NLB exists

Prevention:

- Use ALB (not NLB) for HTTP-based integrations via VPC Link where possible — ALB has SGs and is easier to debug
- Test NLB health independently before attaching VPC Link
- Monitor VPC Link status change events via CloudWatch Events/EventBridge
- Use HTTP API with VPC Link (simpler and cheaper than REST API VPC Link)

Issue 10: API Gateway Request/Response Mapping Template Errors

What happened: API Gateway mapping template (VTL) throwing 400 or 500 errors on valid requests. Integration worked fine but responses are garbled.

How to investigate:

- This typically applies to REST APIs with non-proxy Lambda integrations using Velocity Template Language (VTL)
- CloudWatch execution logs (INFO level): shows the transformed request/response bodies
- Is the VTL template valid? Syntax errors cause silent failures
- Is the Content-Type header correct? VTL templates are selected based on Content-Type
- Are required fields present in the request that the template assumes exist?

How to fix:

- Enable API Gateway execution logs at INFO level — shows pre- and post-transform bodies
- Fix VTL syntax using the console's built-in test feature with sample payloads
- Add null checks in VTL: `#if($input.path('$.field'))` before accessing potentially missing fields
- Use Lambda proxy integration instead — removes mapping templates entirely, Lambda handles raw request/response

Prevention:

- Avoid VTL mapping templates if possible — use Lambda proxy integration for full control in code
- If VTL is required: test every template change with the API Gateway test invoke tool with representative payloads
- Keep VTL templates in version control and require PR review — they're easy to break silently
- Use OpenAPI spec (Swagger import) for API Gateway definitions — keeps mapping templates documented alongside the API

SQS Production Issues

Issue 1: SQS Queue Depth Growing Unboundedly

What happened: SQS `ApproximateNumberOfMessagesVisible` growing into millions.
Consumer cannot keep up with producer.

How to investigate:

- CloudWatch: compare `NumberOfMessagesSent` vs `NumberOfMessagesDeleted` — is deletion rate lower than send rate?
- Is the Lambda consumer throttled? Check Lambda Throttles metric
- Is the consumer throwing errors and not deleting messages? Messages become visible again after visibility timeout

- Is the consumer processing too slowly? Check consumer throughput vs producer throughput
- Is the Lambda batch size too small? Default is 1 message per invocation — very inefficient

How to fix:

- Scale Lambda concurrency (increase reserved concurrency or request limit increase)
- Increase Lambda batch size: up to 10,000 messages per batch for standard SQS, 10 for FIFO
- Enable batch window: `MaximumBatchingWindowInSeconds` to let messages accumulate before triggering Lambda — reduces overhead
- Fix consumer errors causing messages to not be deleted
- Add more consumer instances if EC2/ECS based

Prevention:

- Set CloudWatch alarm: `ApproximateNumberOfMessagesVisible > 1000` and growing → alert
- Design consumers to process at least 2-3× the expected peak send rate
- Set Lambda batch size to 10 (max for standard SQS) for high-throughput queues
- Implement DLQ to capture failed messages rather than letting them block the queue

Issue 2: SQS Messages Processed Multiple Times (Duplicate Processing)

What happened: Lambda consumer processing some messages 2-3 times. Duplicate records appearing in database.

How to investigate:

- SQS Standard guarantees at-least-once delivery — duplicates are by design
- Lambda receives a message, starts processing, but does not complete and delete within the visibility timeout → message becomes visible and is received by another invocation
- Lambda itself retries on failure — same message processed multiple times

- Check: is visibility timeout $\geq$ Lambda function timeout? If Lambda runs for 30s but visibility is 20s, duplicate processing is guaranteed

How to fix:

- Set visibility timeout = $6 \times$ Lambda timeout (AWS recommended ratio)
- Implement idempotency: check a unique message ID in DynamoDB before processing, write the ID after, skip duplicates
- Use Lambda Powertools Idempotency module for Java/Python — handles this with DynamoDB automatically

Prevention:

- SQS Standard: always assume and handle duplicates — design consumers to be idempotent
- Set visibility timeout = $6 \times$ Lambda timeout — absolute minimum requirement
- Use SQS FIFO with message deduplication ID if exactly-once is a strict requirement
- Store processed message IDs in Redis or DynamoDB with TTL = visibility timeout duration

Issue 3: Dead Letter Queue Filling Up — Silent Failures

What happened: DLQ suddenly has thousands of messages. Processing failures happening silently for days. Data loss risk.

How to investigate:

- What error is causing messages to fail? Retrieve a DLQ message and check the payload
- Check original queue's **RedrivePolicy**: **maxReceiveCount** — how many retries before DLQ?
- Lambda CloudWatch Logs: what errors are logged during processing?
- Is it a deployment-related issue? Did a code change break message processing?
- Are DLQ messages all from the same time period (batch incident) or ongoing?

How to fix:

- Fix the root cause error in the consumer
- After fixing, use SQS DLQ Redrive (console or CLI) to move DLQ messages back to source queue for reprocessing
- `aws sqs start-message-move-task --source-arn <dlq-arn> --destination-arn <source-queue-arn>`
- Verify messages process successfully from DLQ before mass-redrive

Prevention:

- CloudWatch alarm: DLQ `ApproximateNumberOfMessagesVisible > 0` → immediate alert — this should never be ignored
- Set `maxReceiveCount` = 3-5 for most workloads — don't retry indefinitely before DLQ
- Add DLQ monitoring to your operational dashboard as a top-priority metric
- Implement structured logging in Lambda consumer with message ID so failures are traceable

Issue 4: SQS FIFO Ordering Not Working as Expected

What happened: Using SQS FIFO for ordered processing but messages from different producers are interleaved unexpectedly.

How to investigate:

- SQS FIFO ordering is per MessageGroupId, not across the entire queue
- If using the same MessageGroupId for all messages: ordering is guaranteed but processing is single-threaded (only 1 active message per group at a time)
- If using different MessageGroupIds: messages in different groups are processed in parallel, independent ordering per group
- Is the consumer setting the correct MessageGroupId per entity?

How to fix:

- Understand the FIFO ordering model: ordering is per group, parallelism is across groups

- For global ordering (rare need): use a single MessageGroupId — accept sequential (slow) processing
- For per-entity ordering (e.g. per-user, per-order): use entity ID as MessageGroupId — parallel across entities, ordered within each

Prevention:

- Design MessageGroupId based on the entity that requires ordering (userId, orderId, accountId)
- Never use a single constant MessageGroupId in a high-throughput system — it's a bottleneck
- Document the FIFO strategy in your ADR: what is the group ID, what ordering guarantee is actually needed
- FIFO queues have a throughput limit of 3,000 messages/sec with batching per API call — design for this ceiling

Issue 5: SQS Visibility Timeout Causing Message Flooding

What happened: Lambda processing times vary — some messages take 1 second, some take 28 seconds. Short visibility timeout (30s) causes messages to reappear before Lambda finishes, triggering duplicate invocations.

How to investigate:

- CloudWatch: `ApproximateNumberOfMessagesNotVisible` is high (messages in flight)
- Lambda Invocations metric is much higher than NumberOfMessagesSent — messages being processed multiple times
- Lambda execution duration P99 approaching or exceeding visibility timeout
- Multiple Lambda log streams showing same MessageId being processed simultaneously

How to fix:

- Set visibility timeout = 6× Lambda max timeout (Lambda max is 15 min, so max visibility = 12 hours)

- Extend visibility timeout programmatically during processing if needed: `ChangeMessageVisibility` API to extend on the fly
- Reduce Lambda max timeout to be realistic for your workload — don't set 15 min if typical execution is 30 seconds

Prevention:

- Set visibility timeout at queue level = $6 \times$ Lambda function timeout — single most important SQS+Lambda setting
- For variable-duration tasks: implement heartbeat: `ChangeMessageVisibility` every $(\text{visibility_timeout} / 2)$ seconds while processing
- Monitor `ApproximateNumberOfMessagesNotVisible` — high values with low queue depth indicate in-flight processing
- Test with realistic message processing time in staging, not just happy-path fast cases

Issue 6: SQS Long Polling Not Configured — High Empty Receive Costs

What happened: SQS costs spike. Analysis shows millions of empty `ReceiveMessage` API calls being charged.

How to investigate:

- Short polling (default): SQS queries only a subset of servers, returns immediately even if no messages (empty response)
- This causes: high API call costs, high CPU usage in consumer, wasted network
- Check `WaitTimeSeconds` on `ReceiveMessage` calls: if 0, short polling is in use
- CloudWatch: `NumberOfEmptyReceives` metric — how many empty polls per period?

How to fix:

- Enable long polling: set `WaitTimeSeconds=20` on `ReceiveMessage` — SQS waits up to 20s for a message before returning
- Set at queue level: `aws sqs set-queue-attributes --queue-url <url> --attributes ReceiveMessageWaitTimeSeconds=20`

- For Lambda trigger: long polling is handled automatically by the Lambda event source mapping

Prevention:

- Set `ReceiveMessageWaitTimeSeconds=20` at queue creation time for all queues with Lambda or polling consumers
- Long polling reduces costs significantly and is better behavior in every dimension — there is no reason not to use it
- For Lambda event source mappings: long polling is automatic — no configuration needed
- Monitor `NumberOfEmptyReceives`: if high and queue is empty most of the time, long polling is not set

Issue 7: SQS Message Retention Period Expiring Before Processing

What happened: Messages being dropped silently. Consumer was down for maintenance. When it came back up, expected messages were gone.

How to investigate:

- Default SQS message retention: 4 days. Maximum: 14 days
- If consumer was down for > 4 days (or your configured retention), messages expired and were deleted
- CloudWatch: `NumberOfMessagesSent` vs `NumberOfMessagesDeleted` — if there's a gap where deletion count dropped but later messages disappeared, retention expiry is the cause
- Check queue attributes: `MessageRetentionPeriod`

How to fix:

- Increase retention period to 14 days for critical queues: `aws sqs set-queue-attributes --attributes MessageRetentionPeriod=1209600`
- Restore lost data from the original source system if possible
- Implement upstream idempotent republishing for recoverable events

Prevention:

- Set `MessageRetentionPeriod=1209600` (14 days) on all production queues — storage cost is negligible
- CloudWatch alarm: consumer downtime > 1 hour → alert, so retention expiry never becomes a risk
- For critical data: persist events to S3/DynamoDB before sending to SQS — SQS is a transport, not a store
- Implement event sourcing for critical business events: events stored permanently, SQS is just the delivery mechanism

Issue 8: SQS Lambda Trigger Stops Processing After Error

What happened: One bad message in SQS queue caused Lambda to fail. Lambda stopped processing all messages. Entire queue frozen.

How to investigate:

- SQS with Lambda trigger: if Lambda throws an error, the entire batch is considered failed
- All messages in the failed batch return to the queue and are retried together
- If one "poison pill" message always causes failure, the same batch is retried indefinitely until `maxReceiveCount` is reached
- Queue is effectively frozen while bad message occupies the batch slot
- CloudWatch: Lambda Errors spiking, queue depth not decreasing

How to fix:

- Enable `ReportBatchItemFailures` in Lambda: return which specific message IDs failed, SQS only retries those, not the whole batch
- Add try/catch per message within the Lambda batch handler — isolate failures
- DLQ configured: bad message gets retried `maxReceiveCount` times then moved to DLQ, unblocking the queue

Prevention:

- Always implement per-message error handling in batch consumers — never let one message poison the entire batch
- Use `ReportBatchItemFailures` response format for all SQS Lambda consumers — returns `{ batchItemFailures: [{ itemIdentifier: messageId }] }`
- Set `maxReceiveCount = 3` and configure DLQ — bad messages are quarantined quickly
- Add message schema validation at the start of Lambda handler — reject malformed messages immediately to DLQ

Issue 9: SQS to Lambda Event Source Mapping Disabled Automatically

What happened: Lambda event source mapping (ESM) for SQS was automatically disabled. Queue stopped being processed. No alert fired.

How to investigate:

- Lambda Console → Configuration → Triggers: check if SQS trigger shows "Enabled" or "Disabled"
- AWS automatically disables an ESM if it encounters repeated errors over an extended period (to prevent billing runup from continuous retry failures)
- CloudWatch: `IteratorAge` or `NumberOfMessagesSent` growing while Lambda invocations are zero
- This happens when Lambda function has been crashing on every invocation for an extended time

How to fix:

- Fix the Lambda function error first
- Re-enable the event source mapping: Lambda Console → Triggers → Enable, or `aws lambda update-event-source-mapping --uuid <id> --enabled`
- Process any backed-up messages after re-enabling

Prevention:

- Set CloudWatch alarm: `Lambda Invocations = 0` while `SQS ApproximateNumberOfMessagesVisible > 0` for > 10 minutes → critical alert

- Fix Lambda errors promptly — don't let functions fail continuously
 - Use Lambda DLQ or Destinations to capture all failures so you don't lose visibility when ESM is disabled
 - Monitor ESM state changes via CloudTrail: `UpdateEventSourceMapping` events
-

Issue 10: SQS Message Attributes Not Being Forwarded to Lambda

What happened: Producer sets SQS message attributes (metadata like messageType, correlationId). Lambda consumer cannot read these attributes. Routing logic broken.

How to investigate:

- Lambda receives SQS event with `messageAttributes` field — check if it's populated
- SQS message attributes are separate from the message body
- For Lambda event source mapping: attributes are forwarded in the event, but only if they were set on the original message
- Check if an intermediate service (SNS, EventBridge Pipe) is stripping attributes during fan-out
- Check the producer code: are attributes being set correctly using SQS MessageAttribute format (not just body fields)?

How to fix:

- Access attributes in Lambda via `event.Records[0].messageAttributes.correlationId.stringValue`
- If attributes are being lost through SNS fan-out: SNS passes SQS message attributes as SNS message attributes — check SNS subscription filter policy is not blocking them
- Move critical routing metadata into the message body as JSON if attributes are unreliable across intermediaries

Prevention:

- Log the full raw event in Lambda during initial development to understand exactly what is received

- For multi-hop architectures (SNS → SQS → Lambda): test attribute propagation end-to-end in staging explicitly
 - Prefer message body JSON over message attributes for critical metadata — body is always forwarded faithfully
 - Document the message schema including attributes in your team's internal API docs
-

That's the complete deep-dive — 10 production issues each across EC2, S3, RDS, Lambda, ASG, ECS, API Gateway, and SQS. Each with investigation, fix, and prevention. Let me know if you want any section expanded or specific scenarios for your Availability/healthcare architecture.

6/5/2026, 10:42:26 AM